AGENTWAY / HARNESS BOOKS

控制面	循环	恢复	先有规矩 再谈聪明
权限	中断	验证	SYSTEM FIRST MODEL SECOND

Harness Engineering

Claude Code 设计指南

一个系统是否可靠，
不在它会不会说，
而在它出了岔子以后，
谁来收拾残局。

CONTROL PLANE / QUERY LOOP / RECOVERY / VERIFICATION

从源码抽象出可控 AI 编程系统的工程原则

agentway.dev

从会用 Agent，到做出 Agent PoC

https://agentway.dev

目录

导读 **1**

序言 Harness **2**

第 1 章为什么需要 Harness Engineering **5**

1.1 问题在于让模型别乱来 5

1.2 Claude Code 的第一层 Harness：受约束的会话系统 5

1.3 第二层 Harness：代理依赖持续循环 6

1.4 第三层 Harness：工具调用必须服从调度 7

1.5 第四层 Harness：最危险的工具，必须配最细的规矩 7

1.6 第五层 Harness：错误属于主路径的一部分 8

1.7 从源码里可以提炼出的第一个原则 8

第 2 章 Prompt 不是人格 **10**

2.1 把 prompt 当成人设，是一种常见误会 10

2.2 从源码看，Claude Code 的 prompt 从一开始就是分层的 10

2.3 Prompt 的真正价值，不在文字本身，而在优先级 11

2.4 Prompt 不是静态文案，它还连接着记忆系统 12

2.5 真正的控制平面，还要考虑缓存与计算成本 13

2.6 用户可以覆盖 prompt，但不能跳过这套结构 13

2.7 为什么说 prompt 在这里更像宪法，而不是台词 14

2.8 从源码里可以提炼出的第二个原则 14

第 3 章 Query Loop **16**

3.1 一个代理系统是否成熟，先看它有没有循环 16

3.2 状态属于主业务 16

3.3 Query loop 的第一职责是治理输入 18
3.4 调用模型只是循环的一段，不是循环本身 19
3.5 心跳必须处理中断，否则它就只是惯性 19
3.6 心跳还必须处理恢复，否则它只是脆弱的重复劳动 20
3.7 停止条件不能只有一个，否则系统会把失败和完成混为一谈 21
3.8 QueryEngine 说明它属于会话生命周期 21
3.9 从源码里可以提炼出的第三个原则 24

第 4 章工具、权限与中断 **25**
4.1 一旦模型开始调用工具，问题的性质就变了 25
4.2 工具调度属于行为宪法的一部分 25
4.3 运行一个工具，真正执行前已经发生了很多事 26
4.4 权限先于能力：Claude Code 没把模型当有天然授权的人 26
4.5 权限结果本身也是一种运行时语义 27
4.6 StreamingToolExecutor 说明中断是一等语义 27
4.7 Bash 为什么永远比别的工具更可疑 29
4.8 工具系统真正保护的，不只是用户，还包括系统自己 31
4.9 从源码里可以提炼出的第四个原则 31

第 5 章上下文治理 **33**
5.1 上下文一多，系统就容易产生一种低级幻觉 33
5.2 `CLAUDE.md` 体系说明，长期指令不能和临场对话混在一起 33
5.3 `MEMORY.md` 是索引，不是日记本 34
5.4 Session memory 说明，短期连续性也不能靠聊天记录硬扛 35
5.5 自动 compact 说明，上下文治理首先是预算治理 36

5.6 `compactConversation()` 说明，摘要要重建可继续工作的上下文 . . 36
5.7 上下文治理的关键是保留工作语义 37
5.8 从源码里可以提炼出的第五个原则 38

第 6 章错误与恢复 **39**

6.1 工程世界最不值得相信的话，就是“正常情况下” 39
6.2 `prompt too long` 是一种必然周期 39
6.3 响应式 compact 说明，恢复的关键在于别把自己逼进死循环 40
6.4 `max_output_tokens` 的处理说明，恢复要以续写为主 41
6.5 auto compact 的失败熔断，说明恢复系统自己也要受治理 41
6.6 compact 自己也会爆，所以连“修复动作”都需要修复策略 42
6.7 abort 语义说明，中断也属于错误恢复的一部分 42
6.8 错误处理真正保护的，是执行叙事的一致性 45
6.9 从源码里可以提炼出的第六个原则 45

第 7 章多代理与验证 **47**

7.1 单代理走到一定程度，问题就不再是“会不会做”，而是“怎么分工” . 47
7.2 forked agent 的第一原则是 cache-safe 47
7.3 状态隔离说明，子代理首先要减少污染 48
7.4 协调者模式说明，synthesis 才是稀缺能力 49
7.5 验证必须独立成阶段，否则“实现完成”很快就会冒充“问题解决” . . 49
7.6 hooks 和任务生命周期说明，子代理不是扔出去就算了 50
7.7 验证不仅针对代码，也针对记忆和建议 52
7.8 多代理真正解决的是不确定性的分区 52
7.9 从源码里可以提炼出的第七个原则 53

第 8 章团队落地 **54**

8.1 个人顺手，不代表团队就能稳定复用 54

8.2 团队第一步，是先把最低边界做清楚 54

8.3 `CLAUDE.md` 的价值，在于稳定、分层、少争议 55

8.4 复用的重点，先是验证定义，再是 skill 数量 56

8.5 skill 更适合作为工作流模块来理解 57

8.6 approval 的重点，是按风险分层 57

8.7 hook 是高级能力，通常不必作为第一步 58

8.8 可复盘轨迹很重要，但要分清基线层和高阶层 59

8.9 从源码里可以提炼出的第八个原则 60

第 9 章 Harness Engineering 十条原则 **61**

9.1 把模型当不稳定部件，不要当同事 61

9.2 Prompt 是控制面的一部分 61

9.3 Query loop 才是代理系统的心跳 61

9.4 工具是受管执行接口 62

9.5 上下文是工作内存 62

9.6 错误路径就是主路径 62

9.7 恢复的目标是继续工作 62

9.8 多代理的意义，是把不确定性分区 62

9.9 验证必须独立，不能让系统自己给自己打分 62

9.10 团队制度比个人技巧重要 63

9.11 最后一句话 63

附录 A 检查清单 **64**

A.1 Agent Runtime 设计清单 64
A.2 Prompt 设计清单 64
A.3 Tool 与 Permission 设计清单 65
A.4 Context 治理清单 65
A.5 Error Recovery 设计清单 66
A.6 Multi-Agent 设计清单 66
A.7 Team 落地清单 66
A.8 Review 问题单 67
A.9 最后一个清单 67

附录 B 图示 **68**

B.1 图一：Claude Code 总体控制面 68
B.2 图二：Query Loop 主循环与恢复分支 69
B.3 图三：Tool Batch Ordering 与 StreamingToolExecutor 70
B.4 图四：Context Sources 与 Compact Rebuild 70
B.5 图五：Coordinator-Worker Flow 与 Verification Separation . . . 70
B.6 图六：团队治理图 70

附录 C 源码地图 **76**

C.1 第 1 章为什么需要 Harness Engineering 76
C.2 第 2 章 Prompt 是控制面，不是人格装修 77
C.3 第 3 章 Query Loop：代理系统的心跳 77
C.4 第 4 章工具、权限与中断 77
C.5 第 5 章上下文治理：Memory、CLAUDE.md 与 Compact 78
C.6 第 6 章错误与恢复 79

C.7 第 7 章多代理与验证 79
C.8 第 8 章团队落地 80
C.9 第 9 章十条原则 80

导读

> 这本书关心的不是“模型会不会写代码”，而是“一个会写代码的模型被放进终端、仓库和团队流程以后，怎样才不会把系统带偏”。

这不是源码注释汇编，也不是产品功能介绍。它关注的是 Claude Code 如何把不稳定模型收束进可持续运行的工程秩序，让控制面、主循环、工具权限、上下文治理、恢复路径、多代理验证与团队制度组织成一套完整骨架。

本书有三个阅读前提：

- 重点不在模型能力，而在 harness 如何组织约束与执行
- 重点不在函数逐条解释，而在运行时结构为什么必须呈现为这种形态
- 重点不在个人技巧，而在这些结构怎样变成团队可以复用的制度

建议阅读顺序：

1. 序言 Harness、终端与工程约束
2. 第 1 章为什么需要 Harness Engineering
3. 第 2 章 Prompt 不是人格，Prompt 是控制平面
4. 第 3 章 Query Loop：代理系统的心跳
5. 第 4 章工具、权限与中断：为什么代理不能直接碰世界
6. 第 5 章上下文治理：Memory、CLAUDE.md 与 Compact 是预算制度
7. 第 6 章错误与恢复：出错后仍能继续工作的代理系统
8. 第 7 章多代理与验证：用分工和验证管理不稳定性
9. 第 8 章团队落地：把一个聪明工具变成可复用制度
10. 第 9 章 Harness Engineering 十条原则
11. 附录 A 检查清单：把原则落成能执行的约束
12. 附录 B 图示：把运行时骨架画出来
13. 附录 C 源码地图：本书各章主要依据哪些文件

如果只想先看总判断，可以直接跳到第 9 章。

序言 Harness、终端与工程约束

这些年，人们喜欢把会写代码的模型叫作智能体。这个词带着明显的乐观色彩，仿佛只要模型能读仓库、调工具、写出像样的 patch，它就可以在工程环境里独立行动。可工程环境有明确后果。终端、文件系统和 Git 历史都不是抽象空间，任何改动都会留下痕迹。

一个只会输出文本的模型，出错时主要带来理解成本。一个能运行命令、写文件、访问网络、修改仓库的模型，出错后留下的是执行结果。目录会变化，进程会中断，配置会损坏，历史会变得难以追踪。到了这一步，核心问题不再是模型是否足够聪明，而是系统是否提供了足够约束。

这本书讨论的，正是这种约束。

我把它叫作 Harness Engineering。这里的 harness 可以理解为一组持续生效的控制结构，用来约束模型在工程环境中的行为边界。对 AI coding agent 来说，没有约束的能力只会扩大事故半径。

这本书也不是一份 Claude Code 源码讲解。源码当然重要，但如果只沿着目录逐个解释，很容易写成注释汇编。那样的内容能说明函数做了什么，却未必能回答系统为什么必须呈现为现在这种结构。要理解 Claude Code 这类系统，光知道有 `queryLoop()`、`compactConversation()`、`runTools()` 还不够。更重要的问题是：为什么一个”会写代码”的系统，最终需要 prompt 分层、权限判定、状态机、compact、恢复分支、subagent 生命周期、verification 阶段和团队制度这些结构?

答案并不复杂，因为模型本身并不稳定。

这个判断未必讨喜，但工程系统不能依赖乐观叙事维持。一个部件如果本质上不稳定，系统就必须围绕这个事实设计；否则，问题只会在事故复盘里集中出现。

Claude Code 值得研究，因为它在实现上保持了明确的工程克制：

- 没有假定模型会持续正确，因此用 query loop 管理状态；
- 没有假定工具调用天然安全，因此用权限和调度约束工具；
- 没有假定上下文越多越好,因此引入 memory、CLAUDE.md、compact 和 session memory；

- 没有把错误视为偶发事件，因此为 prompt too long、max output tokens、中断和 hook 回环设计恢复路径；
- 没有把多代理直接等同于更强能力，因此把 synthesis 和 verification 单独拆开，避免系统自我背书。

这一整套东西，合起来才是 agent。模型只是 agent 里最会说话、也最不稳定的那个部件。

所以这本书有一个始终不变的基本立场：

> Prompt 决定它怎么说话，Harness 决定它怎么做事。

这里说的 harness，不是一层附属工具，也不是对模型能力的情绪化防御。它是模型进入工程环境的前提。缺少这层约束，风险最终会转移给用户、团队和未来的维护者。

这里也先说明一个边界：本书不会附带 Claude Code 源代码，也不会长篇逐段转录源码。原因很简单，就是版权边界。我们能做的，是在合理引用和工程分析的范围内，基于源码结构提炼设计原则、运行机制与方法论判断，而不是把受版权保护的实现文本重新发布一遍。

本书试图做两件事。

第一件，是基于 Claude Code 源码，把真正决定系统可靠性的结构讲清楚。重点在于解释为什么上下文治理必须成为主路径，为什么多代理解决的是职责分区，为什么团队制度必须纳入生命周期节点，而不是简单罗列“这里有 compact”“这里有 subagent”“这里有 hook”。

第二件，是把这些实现背后的判断提炼成更一般的工程原则。具体代码版本会变化，函数名会变化，产品形态也会变化。但只要大家还在尝试把不稳定模型接入真实工作流，某些原则就仍然有效。比如：

- 错误路径要按主路径设计
- 验证必须进入完成定义
- 权限是系统器官，而不是附属功能
- 上下文是资源，不是垃圾桶
- 多代理要靠角色分离，不靠人海战术
- 团队制度比个人技巧重要

如果这些判断成立，那么 Claude Code 更适合作为一份样本。它的价值不在于教人复制一套一模一样的 CLI，而在于展示一个面向真实工程环境的 AI 代理，最终会如何走向更严格的约束结构。

更直接地说，这本书不讨论如何用模型包装出一个”像工程师”的幻觉，而讨论如何在模型并不具备工程师稳定性的前提下，仍然构造出一个可运行的工程系统。

这类工作通常不显眼。回滚、审批、权限、验证、compact、清理孤儿进程都不显眼，但系统能否长期稳定运行，往往取决于这些部分。过度追求”像人一样自然”的代理体验，其常见结果是系统具备了类似人的失误模式，却没有承担后果的能力。

既然如此，就从约束谈起。

接下来九章，讨论的都是这套 Harness 结构如何形成，为什么必须采用这种安排，以及一个团队如何把个人经验沉淀成可以复用的工程制度。

@wquguru
2026.04.01
Claude Code 源码泄漏的愚人节

btw. 您可以在 harness-books.agentway.dev/book1-claude-code 访问在线版，获取更好的阅读体验

第 1 章为什么需要 Harness Engineering

1.1 问题在于让模型别乱来

这些年，人们很喜欢谈智能体。这个词常常带着轻快的预期，仿佛只要模型会写几段代码、会调几个工具，就可以像见习工程师一样在终端里独立工作。可终端和文件系统都带有明确后果。一个会说话的概率分布，一旦能接触 shell、Git、网络和本地文件，问题就从”回答得不够好”变成”执行造成实际破坏”。

所以问题的重点，一直是怎样把它约束成一个可管理的系统。所谓 Harness Engineering，讨论的就是这件事。Harness 是一整套制度化的控制平面，用来处理一个很现实的问题：模型并不天然值得信任。

这个判断未必轻松，但通常有用。一个代理系统要进入真实工程环境，首先要承认自己的核心部件是不稳定的。忽视这一点，问题最后通常会在日志和事故记录里出现。

1.2 Claude Code 的第一层 Harness：受约束的会话系统

如果只看表面，Claude Code 像一个能和用户对话、还能顺手改代码的 CLI。可从实现上看，它一开始就没有把自己当成“裸模型接口”来设计，而是当成一个带有上下文边界、运行时状态和行为规约的会话系统。

这一点从 system prompt 的组织方式就能看出来。

- 在 `src/constants/prompts.ts:175` 开始，系统先定义身份和总任务。
- 在 `src/constants/prompts.ts:186` 开始，补上关于工具、权限、系统提醒和上下文压缩的系统级说明。
- 在 `src/constants/prompts.ts:199` 开始，再补上做任务时的工程约束，比如不要越权改动、不要把验证说成已经完成、不要为了省事发明抽象。

这里值得停一下。很多人谈 prompt，还停留在“你是一个什么样的助手”这种修辞层面。Claude Code 在实现上把 prompt 放进了运行时控制结构里，这些文字用来规定执行边界、失败行为和报告责任。

更重要的是，这个 prompt 采用分段拼装方式。在 `src/constants/prompts.ts:444` 的 `getSystemPrompt()` 里，静态部分和动态部分被明确拆开，memory、language、output style、MCP instructions、scratchpad 等内容按段注入。到了 `src/utils/systemPrompt.ts:28`，系统又把默认 prompt、自定义 prompt、agent prompt 和 append prompt 组织成一套优先级规则。

这说明了一个朴素的工程事实：一个真正可用的代理系统，不能依赖一段“万能提示词”解决所有问题。它必须把控制拆成层，把层次拆成职责。否则，新增提醒和禁令很快就会互相冲突，系统行为也会变得难以预测。

1.3 第二层 Harness：代理依赖持续循环

如果说 prompt 规定了它应该成为什么样的东西，那么 query loop 规定了它实际上如何运行。

Claude Code 的核心不在某个单独的 API 调用，而在 `src/query.ts:219` 开始的 `query()`，以及 `src/query.ts:241` 开始的 `queryLoop()`。这段实现里最重要的一点，是它明确承认代理系统依赖带状态的多轮执行。

在 `src/query.ts:268`，系统把 `messages`、`toolUseContext`、`autoCompactTracking`、`maxOutputTokensRecoveryCount`、`hasAttemptedReactiveCompact`、`pendingToolUseSummary`、`turnCount`、`transition` 等内容放进同一个跨迭代状态里。一个会话系统一旦这样设计，就等于正式承认：上一轮留下的问题会进入下一轮，系统必须有能力继续处理。

这是 Harness 思维的核心。真正的问题在于系统能不能在连续多轮里保持行为一致：

- 有没有预算概念
- 有没有恢复概念
- 有没有上下文膨胀后的自救机制
- 有没有在工具调用失败后继续推进任务的能力

缺少这些结构，所谓智能体就只是一个不稳定的执行者。

在 `src/query.ts:365` 往后，这个循环还会在每轮调用前处理消息裁剪、tool result budget、history snip、microcompact、context collapse、autocompact 等内容。

实现细节虽然很多，但共同指向一点：Claude Code 在调用发生前就尽量把控制权收回到运行时一侧。

这也是为什么 Harness Engineering 不能被看作 prompt engineering 的附属品。前者关心状态机，后者关心措辞。措辞当然重要，但状态机决定系统行为最终由谁负责。

1.4 第三层 Harness：工具调用必须服从调度

一个模型如果只能输出文本，顶多让人觉得它有时说得太满。可一旦它能调用工具，系统风险就立刻从修辞风险变成执行风险。这时候最重要的问题是：谁决定工具怎么跑。

Claude Code 给出的答案很直接。运行时会根据工具属性决定并发还是串行。

在 `src/services/tools/toolOrchestration.ts:19` 的 `runTools()` 里,工具调用先经过 `partitionToolCalls()` 分组。到了 `src/services/tools/toolOrchestration.ts:91` 系统会读取工具 schema, 并调用 `isConcurrencySafe()` 判断一个工具是否适合并发执行。能并发的归成一批,不能并发的按顺序一个个来。并发路径里,context modifier 会先缓存,再按原始 block 顺序回放,见 `src/services/tools/toolOrchestration.ts:31` 到 `:63`。

这件事很有代表性。它说明 Claude Code 没有把工具当成模型能力的自然延伸，而是当成需要调度纪律的受管执行单元。缺少调度纪律的工具系统，只会把模型的不稳定性放大到外部世界。

并发如果不受约束，就会扩大事故半径。Claude Code 在这里采取了偏保守的策略。在会碰到文件、终端和权限的场景里，这种保守通常更可靠。

1.5 第四层 Harness：最危险的工具，必须配最细的规矩

在所有工具里，Bash 最值得警惕。因为它几乎不受领域边界约束，可以直接接触文件、进程、网络和 Git 仓库, 还会带上重定向、管道等复杂 shell 语义。一个系统如果对 Bash 过度信任，后果通常会很具体。

Claude Code 对 Bash 的态度，可以在 `src/tools/BashTool/prompt.ts:42` 往后看得很清楚。这里写了一整段操作规约，尤其是围绕 git 和 PR 的那部分：不要乱改 git config，不要跳过 hooks，不要随手 `git add .`，不要在 pre-commit 失败后用 `--amend` 把上一条提交也搭进去，不要在没有明确要求时提交，更不要默认 push。

写到这个地步，有人会觉得它过于细碎。但高风险接口通常就需要高密度约束。Bash 一旦进入真实工作流，很多规则都必须明确写出来。

Harness Engineering 的一个重要原则，就是把高风险能力包装成高约束能力。能力越强，控制越细。原因很简单：外部世界不会因为模型语气坚定，就自动原谅一次错误执行。

1.6 第五层 Harness：错误属于主路径的一部分

很多软件把失败路径看成例外，把成功路径看成正文。代理系统不能这样做。因为代理系统的失败不是偶发性的，它是一种稳定存在。模型会超 token，会触发 `prompt too long`，会撞上 `max_output_tokens`，还会遇到工具拒绝、用户打断、hook 阻塞、API 重试等各种中断。要是这些情况都只在最后用几个 `catch` 打发一下，那系统表面上在运行，实际上只是不断把麻烦往后滚。

Claude Code 在 query loop 里没有这样处理。光看 `src/query.ts:453` 往后关于 autocompact 的处理，以及 `src/query.ts:592` 往后对上下文上限和阻断逻辑的注释，就能看出它把失败当作会持续发生的结构性条件来处理。

这也是 Harness 和普通助手的重要差别之一。普通助手常见的设计逻辑是先回答，错了再道歉。Harness 更强调先约束，再执行；即使出错，也要按恢复路径处理，而不是靠临场发挥补救。

一个会道歉的系统，不一定成熟。一个知道何时不该开始、何时该重试、何时该中止、何时该准确汇报失败的系统，才更接近成熟。

1.7 从源码里可以提炼出的第一个原则

到这里，第一章其实只想说一件事：

> 代理系统的关键能力是约束执行。

Claude Code 的源码在几个关键位置都指向同一个结论：

- `constants/prompts.ts` 说明 prompt 是控制平面的一部分，而不是人格装饰
- `utils/systemPrompt.ts` 说明系统行为必须有清楚的分层优先级
- `query.ts` 说明代理运行依赖持续的循环状态，而不是单次问答
- `services/tools/toolOrchestration.ts` 说明工具调用必须服从调度纪律
- `tools/BashTool/prompt.ts` 说明高风险工具必须伴随高密度约束

把这些放在一起看，就会发现 Harness Engineering 并不神秘。它只是坚持几条常被忽视的工程常识：

- 模型会犯错
- 工具会扩大错误后果
- 上下文会膨胀
- 状态会污染下一轮
- 用户会打断你
- 失败会反复出现

既然如此，系统就不能靠“聪明”维持秩序，只能靠结构维持秩序。结构不像聪明那样显眼，但通常更可靠。

下一章要谈的，是这套结构里最容易被误解的一层：system prompt。很多人把它看成人设文本，本书会说明它更接近操作系统里的规章制度。人设可以改善观感，规章才能约束机器。

第 2 章 Prompt 不是人格，Prompt 是控制平面

2.1 把 prompt 当成人设，是一种常见误会

很多人一说起 system prompt，首先想到的是一段熟悉的话术：你是谁，你擅长什么，你应该温柔、专业、简洁，最好再有一点稳定的人格。对于只负责聊天的系统，这种理解倒也够用；但对一个要读文件、调工具、动 shell、处理权限、跨轮执行的代理系统来说，这种理解明显不够。

原因很简单。人设描述解决的是“它像什么”，控制平面解决的是“它能做什么、什么时候做、做错了怎么办、谁来兜底”。两者不在同一层。一个系统可以有讨人喜欢的人设，同时在执行层面缺少规矩。那种系统出事时往往会显得很真诚，因为它很会道歉。但道歉并不能替代运行时设计。

Claude Code 的实现恰好说明了这一点。它的 system prompt 是一组分层拼装的行为区块。换句话说，这里的 prompt 更接近一套运行时协议，而不是一篇人物小传。

2.2 从源码看，Claude Code 的 prompt 从一开始就是分层的

在 `src/constants/prompts.ts:444` 的 `getSystemPrompt()` 里，Claude Code 返回的是一个由多个 section 组成的数组，而不是一段完整字符串。这个细节很重要。因为一旦 prompt 变成多个块，系统就正式承认它内部包含一组职责不同的约束。

这些 section 至少包含几类东西。

首先是身份和总任务说明。在 `src/constants/prompts.ts:175`，系统说明自己是一个交互式代理，要用可用工具帮助用户完成软件工程任务。这里同时嵌入了一些安全约束，比如不要乱猜 URL。

然后是系统级规则。在 `src/constants/prompts.ts:186` 开始，系统明确规定：

- 用户能看见的是哪些文本
- 工具调用可能触发权限审批
- 用户拒绝后不能机械重试
- tool result 和 user message 里可能混入 system-reminder
- 上下文会被自动压缩

这些内容有一个显著特征：它们并不关心模型“像不像一个聪明助手”，而是关心它是否是一个守规矩的执行体。这就是控制平面的语气，它的核心任务是定义边界。

再往下，在 `src/constants/prompts.ts:199` 开始，是做任务时的工程性指令：不要随意增加需求，不要越权优化，不要为了让结果显得完整而隐瞒验证失败，不要在没有必要时制造抽象。这些内容看起来像写作风格要求，其实它们和工程约束绑得很紧。一个会自动“顺手优化一切”的模型，从产品角度看也许很热情，从工程角度看则相当危险。

所以，从源码结构上就能看出来：Claude Code 的 prompt 要解决的是如何让模型在复杂运行时里遵守边界。

2.3 Prompt 的真正价值，不在文字本身，而在优先级

如果 prompt 只是写在那里，还不够说明问题。真正决定它是否属于控制平面的，是系统是否给它定义了严格优先级。

这一点可以看 `src/utils/systemPrompt.ts:28` 开始的 `buildEffectiveSystemPrompt()`。这段代码把 prompt 的来源明确排成一条链：

1. override system prompt
2. coordinator system prompt
3. agent system prompt
4. custom system prompt
5. default system prompt

最后还会统一拼接 `appendSystemPrompt`。

这个设计很说明问题。它表明 Claude Code 并不相信“默认 prompt 一劳永逸”。相反，它承认系统里存在多种语境：

- 协调者模式需要自己的系统行为
- agent 模式需要自己的职责说明
- 用户可以通过 CLI 覆盖或追加 prompt
- 默认 prompt 只是没有更高优先级时的基线

更朴素地说，成熟系统不会迷信唯一版本的 prompt。它会把 prompt 看成一个有层级的配置系统，让不同职责在不同上下文里生效。

这里还有一个很值得注意的细节。在 `src/utils/systemPrompt.ts:99` 往后，系统对 proactive mode 做了特殊处理：如果 agent prompt 和 proactive mode 同时存在，agent prompt 不再替换默认 prompt，而是附加在默认 prompt 之后。这个决定本身就很说明问题。它意味着系统知道，有时候默认约束不能丢，新增 agent 只能在默认约束之上叠加领域行为，而不能把整套纪律换掉。

可以把它理解为一套通用制度外加岗位说明书。岗位说明书可以补充职责，但不能直接冲掉底层制度，否则系统很快就会各自为政。

2.4 Prompt 不是静态文案，它还连接着记忆系统

如果说前面这些内容已经像一套运行时说明书，那么看到 Claude Code 如何处理 memory 和 `CLAUDE.md` 后，就会更清楚地意识到：这里的 prompt 已经是整个上下文治理入口，而不只是“写给模型看的一段话”。

在 `src/utils/claudemd.ts:1153` 的 `getClaudeMds()` 里，系统会把 project instructions、local instructions、team memory、auto memory 等不同来源的内容整理成统一格式，再拼接进 prompt 相关上下文中。这里连每种内容的来源说明都写得很细，比如这是项目级指令、用户私有项目指令、共享 team memory，还是跨会话持久化的 auto memory。

而在 `src/memdir/memdir.ts:187` 开始的 `buildMemoryLines()` 里，系统连“如何保存记忆”这件事都变成了 prompt 的一部分。它会明确告诉模型：

- memory 是文件化持久系统
- `MEMORY.md` 是索引，不是正文
- 要如何写 frontmatter
- 哪些信息不该保存
- plan 和 task 不该被误用成 memory

这件事非常关键。它把 prompt 的职责从“约束当前行为”扩展到了“约束未来知识的沉淀方式”。这已经超出了通常意义上的提示词，更接近一份写给运行时参与者的知识治理协议。

换句话说，Claude Code 不只是用 prompt 规定“这一轮怎么说话”，还用 prompt 规定“长期记忆如何形成”。一个系统只要走到这一步，它的 prompt 就不可能再只是语气问题，而必然进入制度问题。

2.5 真正的控制平面，还要考虑缓存与计算成本

多数人理解 prompt 时，很少会想到性能。常见想法是 prompt 只是喂给模型的文本，写好即可。Claude Code 的实现更务实：prompt 同时也是计算成本。它越复杂、变化越频繁，缓存命中就越差，系统运行就越贵、越慢。

在 `src/constants/systemPromptSections.ts:16` 往后，系统把 prompt section 区分成两类：

- 可缓存的 `systemPromptSection`
- 会打破缓存的 `DANGEROUS_uncachedSystemPromptSection`

而 `resolveSystemPromptSections()` 会优先从缓存里拿已经计算过的内容，只在必要时重算。到了 `clearSystemPromptSections()`，系统又会在 `/clear` 或 `/compact` 之后清空这些状态。

这件事看起来像优化，实际上同样属于控制平面。一个真正可运行的 prompt 系统，不可能只考虑表达能力，而不考虑它对吞吐、延迟和缓存的影响。Claude Code 在 `getSystemPrompt()` 里甚至把静态部分和动态部分用 boundary 显式分开，见 `src/constants/prompts.ts:560` 往后。这说明它在设计时已经承认：有些内容在会话中相对稳定，有些内容会逐轮变化，二者不能混在一起消耗缓存。

一个工程系统只要开始关心“哪部分 prompt 会导致缓存失效”，它就已经不再把 prompt 当作文案创作。文案追求完整表达，控制平面追求可治理、可复用、可预测的行为成本。两者关注的问题不同。

2.6 用户可以覆盖 prompt，但不能跳过这套结构

Claude Code 并没有把用户锁死在默认 prompt 上。相反，CLI 明确支持覆盖和追加。

在 `src/main.tsx:1342` 往后，系统处理 `--system-prompt`、`--system-prompt-file`、`--append-system-prompt`、`--append-system-prompt-file` 这些选项。也就是说，用户当然可以带着自己的规约来。

但这里有个关键点。系统虽然允许覆盖和追加，却仍然坚持用统一的 `buildEffectiveSystemPrompt()` 做最终装配。这说明它允许自定义，但不放弃秩序。用户可以改内容，系统仍然保留结构。

没有结构的可定制，最后往往会退化成另一种随意。今天加一段，明天减一段，后天某个 agent 又替换掉基线约束，系统行为就会越来越像临时口头通知。Claude Code 的选择是让用户修改，但修改必须发生在既定优先级和分层机制里。

2.7 为什么说 prompt 在这里更像宪法，而不是台词

如果把前面各节放在一起看，可以得到一个相当明确的结论：

Claude Code 的 prompt 更像宪法。

所谓台词，是给角色在场上说的；所谓宪法，是规定权力边界、责任关系和例外情况如何处理。Claude Code 的 prompt 更接近后者，因为它满足了几个结构条件：

- 它分层，而不是一块写到底
- 它有优先级，而不是谁后写谁说了算
- 它与 memory、CLAUDE.md、agent instructions、MCP instructions 一起组成完整控制平面
- 它有缓存和动态 section 机制，不是随手拼一段文本
- 它和 runtime 紧密耦合，而不是游离于系统之外的装饰物

这也是为什么“写一个好 prompt”单独拿出来时价值有限。更重要的问题是：prompt 在系统里处于什么位置，它和哪些模块配合，它是否参与权限、状态、上下文和长期记忆的治理。如果不回答这些问题，所谓好 prompt 往往只是在某个顺利场景里暂时成立。

2.8 从源码里可以提炼出的第二个原则

这一章最后可以归纳成一句话：

> Prompt 的价值，在于它是否被纳入一套清楚的控制结构。

Claude Code 的源码在几个地方共同证明了这一点：

- `constants/prompts.ts` 把 prompt 写成分段控制结构，而不是一段统一宣言
- `utils/systemPrompt.ts` 明确规定了 prompt 来源的优先级
- `utils/claudemd.ts` 把项目级和长期记忆内容纳入上下文装配
- `memdir/memdir.ts` 用 prompt 规定了长期记忆的保存规则
- `constants/systemPromptSections.ts` 则把 prompt 进一步变成可缓存、可失效、可按段重算的运行时对象

所以，一个成熟代理系统里的 prompt，不该被理解成“让模型入戏的开场白”。它更像一套运行中的制度文本。制度文本当然也可以写得清楚，但最重要的部分始终是约束力。

下一章要讨论的，是另一根更硬的骨头：query loop。因为再好的控制平面，最后都要落到执行循环里。prompt 规定边界，循环决定命运。一个系统最终会成为什么样子，往往体现在它每一轮如何继续、如何中断、如何恢复的那套状态机里。

第 3 章 Query Loop：代理系统的心跳

3.1 一个代理系统是否成熟，先看它有没有循环

如果把一个会写代码的模型看成代理系统，最容易犯的错误，就是把它想象成一个加强版问答接口。用户发来一句话，模型输出一个结果，事情就算办完。这种想法并非毫无来由，因为很多大模型产品确实这样工作。但只要系统开始调用工具、跨轮执行、处理中断、保存状态、重试失败、压缩上下文，这种“一问一答”的理解就会迅速失效。

Claude Code 的实现没有犯这个错误。它从结构上明确承认：代理依赖一段持续的、有状态的执行过程。

这一点在 `src/query.ts:219` 的 `query()` 和 `src/query.ts:241` 的 `queryLoop()` 里表现得很明显。前者只是壳，真正重要的是后者。`queryLoop()` 不是把模型调用包在一个 `try/catch` 里就结束。它维护了一套跨迭代状态，先处理一系列前置治理动作，然后进入模型流式阶段；等模型返回后，再决定是进入工具执行、恢复、压缩、继续下一轮，还是直接终止。

这意味着 Claude Code 的核心是维持一个会话内的执行秩序。这里的关键名词是 lifecycle。一个系统是否能被称为 agent，往往不取决于它会不会说，而取决于它能不能在几轮之后仍然知道自己在做什么。

3.2 状态属于主业务

很多系统在设计之初，都倾向于把状态看成包袱，仿佛无状态才更优雅。对代理系统来说，这种偏好作用有限。只要它进入真实工作流，状态就会自然出现。忽视状态，并不能消除状态，只会让它以更难管理的方式返回。

Claude Code 在这里的态度很直接。在 `src/query.ts:203` 到 `:217`，系统把 query loop 的可变状态定义得很清楚：

- `messages`

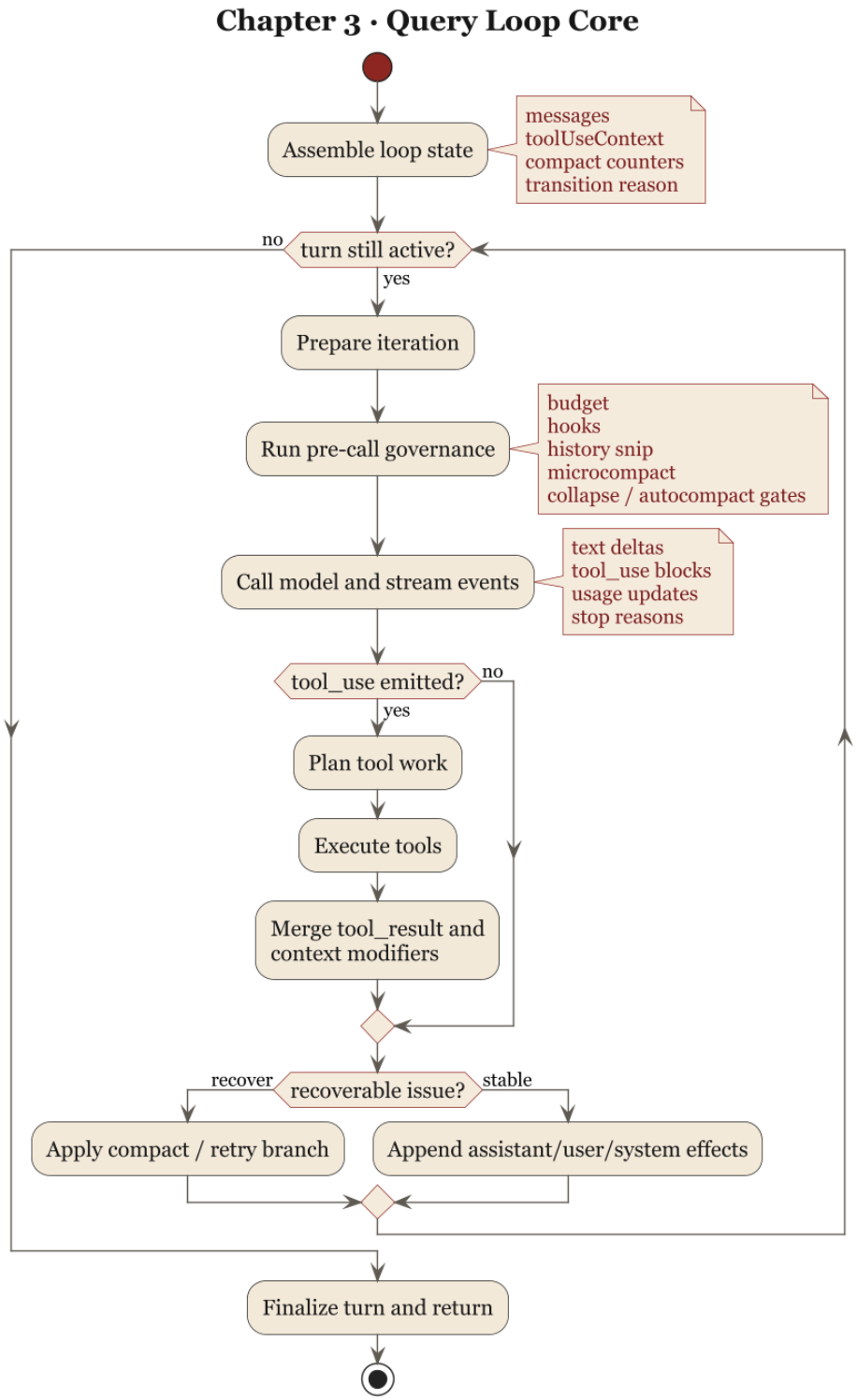

图 1: Claude Code Query Loop Core

- `toolUseContext`
- `autoCompactTracking`
- `maxOutputTokensRecoveryCount`
- `hasAttemptedReactiveCompact`
- `pendingToolUseSummary`
- `stopHookActive`
- `turnCount`
- `transition`

到了 `src/query.ts:268`，这些状态在每次 query loop 启动时被整体装配成一个 `State` 对象，并在后续各个 continue 分支里整体更新。

这一点很重要。Claude Code 没有把恢复、压缩、预算、hook、turn 计数散落在局部变量和布尔开关里，而是承认它们共同构成了“本轮结束后下一轮如何继续”的基础。它把状态当作心跳的一部分。

这就是成熟代理系统和一次性脚本的区别。脚本只关心这一步有没有跑完，代理系统还要关心：这一步失败之后，下一步能不能继续承接前面留下的状态。

3.3 Query loop 的第一职责是治理输入

从外部看代理系统，很多人会以为它的核心动作是“调用模型”。但在工程上，真正重要的常常是模型调用之前那一长串整理工作。Claude Code 在 `queryLoop()` 里把这件事写得很清楚。

在正式进入模型流之前，系统会先做这些事：

- 启动相关 memory 的预取，见 `src/query.ts:297`
- 预取 skill discovery，见 `src/query.ts:323`
- 截取 compact boundary 之后的有效消息，见 `src/query.ts:365`
- 应用 tool result budget，见 `src/query.ts:369`
- 进行 history snip，见 `src/query.ts:396`
- 进行 microcompact，见 `src/query.ts:412`
- 进行 context collapse，见 `src/query.ts:428`
- 最后才尝试 autocompact，见 `src/query.ts:453`

这串顺序本身就是一种架构声明。它告诉读者，Claude Code 把“上下文治理”放在“模型推理”之前。也就是说，它不把从混乱中整理秩序的责任交给模型，而是先由运行时完成治理，再把更干净的输入交给模型。

这件事很重要，因为很多系统恰恰相反：先把大量上下文塞进去，再寄希望于模型自己判断什么重要，什么不重要。那种做法看似省事，实际上是在把运行时应承担的责任转嫁给概率分布。

Claude Code 的做法更接近传统工程流程：先整理现场，再开始执行。它不追求潇洒，但通常更稳妥。

3.4 调用模型只是循环的一段，不是循环本身

等前面的治理工作都做完，Claude Code 才进入模型调用阶段。这个阶段出现在 `src/query.ts:652` 往后。这里有个值得专门指出的细节：系统会进入 `for await` 流式消费模型输出，而不是同步拿一个完整结果回来。

这意味着模型输出在 Claude Code 里是一串事件流，而不只是“最终答案”。事件里可能包含：

- assistant 文本
- `tool_use` block
- usage 更新
- stop reason
- API 错误

这一点在 `src/query.ts:826` 往后尤其明显。系统会把 assistant message 存起来，提取其中的 `tool_use` block，决定是否需要 follow-up，还可能边流边把工具送给 `StreamingToolExecutor`。

从工程角度看，这是一种根本性的变化。一旦把模型输出当成事件流，系统架构就不再只是“请求-响应”，而更像“驱动-调度-反馈”的过程。流式输出的意义，也不只是更早看到几个字，而是允许运行时在模型尚未完全结束之前，就开始安排下一步执行。

这也是为什么前面说 query loop 才是代理系统的心跳，而不是模型调用本身。模型调用只是心跳中的一次收缩，真正维持系统运行的是整套循环：输入如何收进来，流如何消费，工具如何调度，失败如何恢复，何时继续下一轮。

3.5 心跳必须处理中断，否则它就只是惯性

一个真正的心跳，不只是能持续跳动，还必须能在必要时停下来。停不下来，系统就只剩惯性。

Claude Code 对中断的处理写得很实在。在 `src/query.ts:1011` 往后，系统会优先处理 streaming abort。如果启用了 `streamingToolExecutor`，就必须先消费剩余结果，生成 synthetic tool_result，避免已经发出的 `tool_use` 没有配套结果；否则，就用 `yieldMissingToolResultBlocks()` 主动补全中断说明。

这背后有一个很基础的工程原则：只要系统向外承诺了一段执行，就要在中断时把账补平。不能因为用户打断了，就假装前面的几个 `tool_use` 从未发生。外部系统、UI 和 transcript 都需要一致的因果链，哪怕结果是“中断了”，也必须中断得完整。

这件事之所以重要，是因为代理系统一旦进入多工具、多轮次状态，外部世界对它的要求就不只是“有没有最终答案”，而是“它留下的轨迹能不能被解释”。不能解释的执行轨迹，迟早会变成运维问题、审计问题，或者变成团队里谁也说不清楚的长期隐患。

所以，处理中断是 runtime 的基本责任。已经开始的动作需要有交代，哪怕交代的是“没做完”。

3.6 心跳还必须处理恢复，否则它只是脆弱的重复劳动

如果说中断是外部世界打进来的意外，那么恢复就是系统内部预留的余量。没有恢复能力的循环，不管表面多整洁，最后都会暴露出同一个问题：它把幸运当成了设计。

Claude Code 对恢复的处理是层层递进的，而不是简单重试。最典型的是 prompt-too-long 和 max-output-tokens。

在 `src/query.ts:1065` 往后，系统会先判断最后一条 assistant message 是否是被 withheld 的 `prompt too long`。如果是，先试图让 context collapse 把积压的 collapse 提交出去（见 `:1086` 到 `:1116`）；如果还不够，再进入 reactive compact（见 `:1119` 到 `:1166`）。换句话说，系统会按成本和破坏性从低到高，逐层尝试恢复。

对 `max_output_tokens` 的处理也一样。在 `src/query.ts:1185` 往后，系统先尝试提升 token cap；如果还不行，再生成一条 meta message，让模型从被截断处继续往下做，而不是先道歉、先总结、先写一段漂亮的空话。

这很能说明 Claude Code 的设计态度。它把恢复看成运行时主路径的一部分，而不是模型失败后的礼貌动作。恢复的意义，在于给系统一个继续工作的机会。在真实工程里，继续工作通常比维持表面上的礼貌更重要。

3.7 停止条件不能只有一个，否则系统会把失败和完成混为一谈

普通问答系统的停止条件比较简单：有回答就结束。代理系统不能这么偷懒。因为一个会话里，出现“当前轮结束”并不等于“任务完成”，更不等于“系统成功”。

Claude Code 的 query loop 至少区分了这些情况：

- streaming 正常完成但有 `tool_use`，需要 follow-up
- 没有 `tool_use`，进入 stop hooks 和可能的后续判定
- 被用户中断
- 触发 prompt-too-long 恢复
- 触发 max-output-tokens 恢复
- stop hook 阻塞导致重进循环
- API 错误直接返回

这可以从 `src/query.ts:1062` 往后一直看到 `:1305`。尤其是 stop hooks 那段，在 `:1267` 到 `:1305`，系统不仅处理 hook，还专门防止“compact 后仍然太长，再被 hook 阻塞，再继续 compact”的死循环。

这个地方很值得注意。许多系统只有一种朴素想法：失败了就重试。Claude Code 则承认，重试本身也是一种需要被管理的行为。系统必须知道为什么重试、已经试过什么、哪些保护状态不能被重置、哪些情况会导致无限循环。正是这些判断，把一个“会继续试”的系统和一个“知道什么时候不该再试”的系统区分开了。

3.8 QueryEngine 说明它属于会话生命周期

如果 `queryLoop()` 还不足以说明问题，那么 `QueryEngine` 的存在就更直接了。

在 `src/QueryEngine.ts:176` 开始，源码明确写着：

> QueryEngine owns the query lifecycle and session state for a conversation.

这句话已经把整章的重点说得很明确。`QueryEngine` 管理的是一个 conversation 的 query lifecycle，而不是某一次调用。`src/QueryEngine.ts:180` 还专门说明：一个 `QueryEngine` 对应一个 conversation，每次 `submitMessage()` 都是在同一个 conversation 里开启新一轮 turn，状态会持续保存。

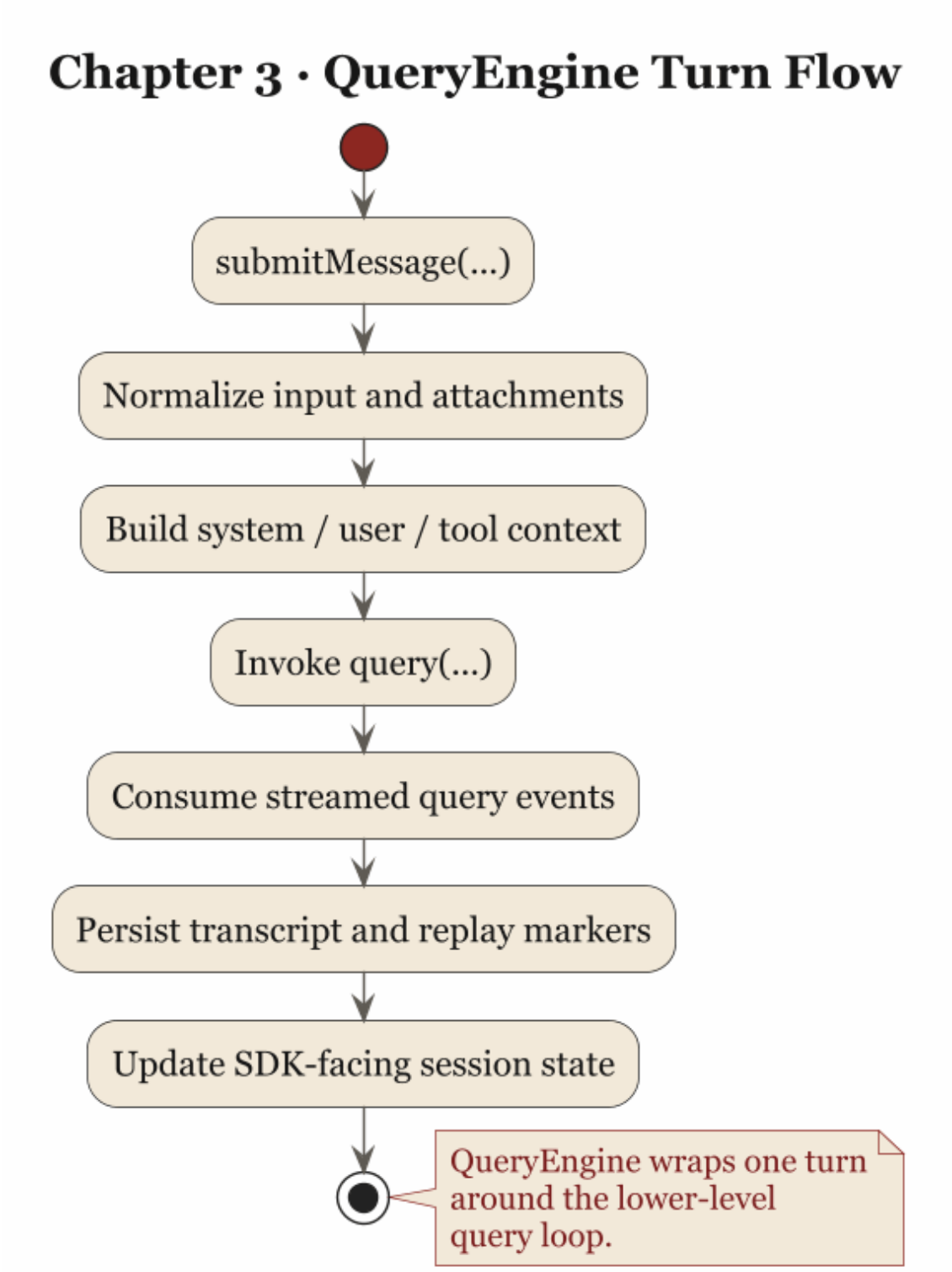

图 2: Claude Code QueryEngine Turn Flow

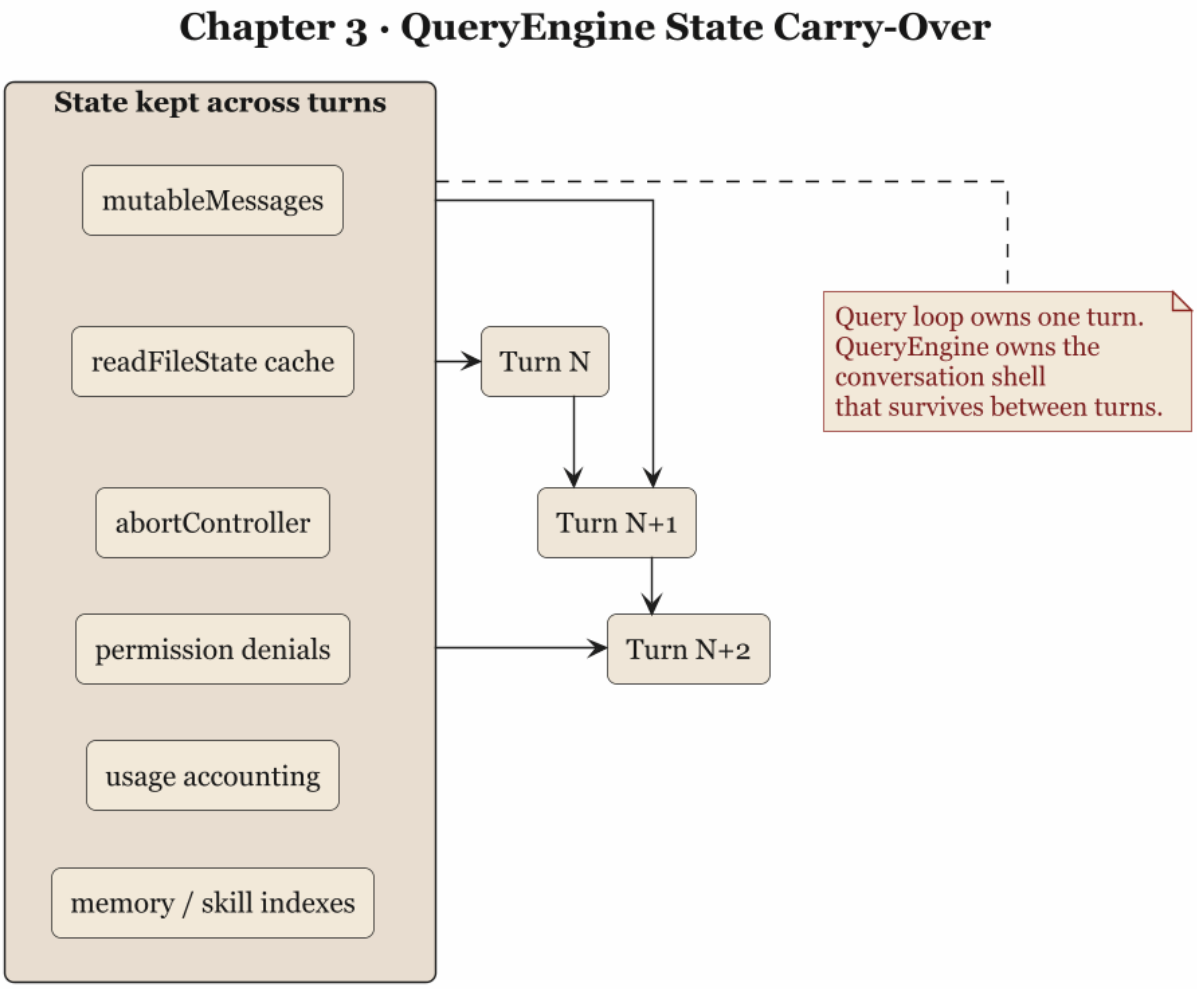

图 3: Claude Code QueryEngine State Carry-Over

到了 `src/QueryEngine.ts:675` 往后，`QueryEngine` 把准备好的 `messages`、`systemPrompt`、`userContext`、`systemContext`、`toolUseContext` 一起交给 `query()`，再把 assistant、user、compact boundary 等消息写回 transcript。

这说明 query loop 是会话系统真正的执行中心。外层的 UI、SDK、session persistence 都围着它转。要理解 Claude Code 的设计，不能只看它有哪些工具，也不能只看它 prompt 写了什么，最终还是得看这个循环如何把前面的约束落实成连续行为。

3.9 从源码里可以提炼出的第三个原则

这一章最后可以收敛成一句话：

> 代理系统的核心能力，是维持可恢复的执行循环。

Claude Code 的源码在几个关键点共同支持这个判断：

- `query.ts` 用显式 `State` 管理跨轮执行状态，而不是把一切寄托在局部变量上
- 模型调用前有大段输入治理逻辑，说明运行时先于推理
- 流式消费把模型输出当事件流，而不是当最终文案
- 中断路径会补齐 synthetic tool_result，说明系统关心因果闭环
- prompt-too-long、max-output-tokens、stop hooks 都走明确恢复分支，说明失败是主路径的一部分
- `QueryEngine.ts` 明确把 query lifecycle 当作 conversation 的所有权对象

这意味着一个成熟 agent 的“心跳”至少要满足几个条件：

- 它有明确的跨轮状态
- 它能治理输入，而不只是被动消费输入
- 它能流式地承接模型输出
- 它能补齐中断后的执行账本
- 它能区分完成、失败、恢复和继续

缺少这些结构的系统，也许仍然能做出漂亮 demo，但它们更接近一次性表演，而不是运行时。表演当然有价值，只是不能替代秩序。

下一章要讨论的，是这套心跳最直接碰到外部世界的地方：工具、权限与中断。前面这一章解释了循环为什么存在，下一章要继续说明，循环一旦拥有工具，为什么必须学会克制。

第 4 章工具、权限与中断：为什么代理不能直接碰世界

4.1 一旦模型开始调用工具，问题的性质就变了

只会输出文本的模型，出错时主要增加沟通成本。它说错了，可以不信；它总结得糟，可以重问。可一旦模型开始调用工具，问题的性质就变了。因为工具不是意见，工具是动作。动作会留下结果，结果会接触真实世界。

这件事在 shell 上最容易看清。一个模型如果把一段解释写错了，影响通常还停留在理解层面；可要是它运行了一条不该运行的命令，文件会被删掉，进程会被中止，Git 历史也会变得难以收拾。能力增强往往伴随后果增强。

所以，工具系统最重要的问题是：谁来约束这些工具。Claude Code 对这个问题的回答，是把工具变成受管执行接口，避免让模型直接伸手去碰世界。

4.2 工具调度属于行为宪法的一部分

Claude Code 在 `src/services/tools/toolOrchestration.ts:19` 的 `runTools()` 里，先做了一件很有代表性的事情：不直接执行一串 `tool_use`，而是先按并发安全性分批。

在 `src/services/tools/toolOrchestration.ts:91` 的 `partitionToolCalls()` 里，系统会先读取工具的 `inputSchema`，再调用 `isConcurrencySafe()` 判断这类调用是否适合并发。如果适合，就把它们归入并发批次；如果不适合，就拆成串行单元。

这看上去像性能优化，实际更接近一致性设计。一个工具系统一旦允许并发，就必须回答一个老问题：上下文变化由谁决定、按什么顺序生效。Claude Code 在并发路径里没有让最先完成的工具抢先改上下文，而是在 `toolOrchestration.ts:31` 到 `:63` 先缓

存 `contextModifier`，再按原始 block 顺序回放。也就是说，即便执行是并发的，语义上的上下文演化仍然保持确定顺序。

这是一种典型的工程保守。它的前提是：并发可以提高吞吐，但不能破坏因果秩序。工具如果只会跑得更快，却不能保证上下文一致性，就会替系统制造另一种随机性。

成熟代理系统不会迷信并发。它会把并发当成需要证明自身无害的例外，而不是默认自由。Claude Code 在这里显然把问题扩散速度考虑得很充分。

4.3 运行一个工具，真正执行前已经发生了很多事

很多人以为 `tool_use` 一旦出现，下一步自然就是执行。Claude Code 的实现说明，真正靠谱的系统不会这么草率。

在 `src/services/tools/toolExecution.ts:30` 之后，`runToolUse()` 所依赖的执行逻辑，已经把 permission、hooks、telemetry、synthetic error materialization 等能力接进来了。即使不追每个细节，只看整体结构也能发现：工具执行在 Claude Code 里是一段完整的流程，包含：

- 前置校验
- 执行中事件
- 执行后修正
- 失败补偿

这说明工具在这里的地位，和普通库函数并不一样。库函数默认属于程序内部，调用者自己承担后果；工具则属于模型与外部世界之间的接口，所以系统不能假设调用者具备稳定判断。换句话说，工具执行周围之所以需要这么多包裹层，是因为调用者本身就是最不稳定的变量。

从设计哲学上说，这一点很重要：工具不应该被建模为“模型能力的延长线”，而应该被建模为“需要运行时代为管理风险的外部能力”。一旦接受这一点，permission、hooks、interrupt、synthetic result 这些结构就更像常识，而不是负担。

4.4 权限先于能力：Claude Code 没把模型当有天然授权的人

Claude Code 的权限入口，在 `src/hooks/useCanUseTool.tsx:27` 往后。`CanUseToolFn` 的存在本身已经说明一件事：工具是否允许执行，并不由模型自己说了算，而

要交给权限判定链。

在 `useCanUseTool()` 里，系统不会因为模型提出了一个工具请求，就默认执行。相反，它会先调用 `hasPermissionsToUseTool(...)` 做权限判定，见 `useCanUseTool.tsx:37`。返回结果会分成 `allow`、`deny` 或 `ask`。这一点看上去平常，其实很关键。因为真正成熟的权限系统，除了“能”和“不能”，还要承认第三种状态：系统自己也不该替用户做决定。

到了 `useCanUseTool.tsx:64` 往后，这条链继续分出不同路径：

- `deny`：直接拒绝
- `ask`：进入协调器、swarm worker、classifier 或交互式审批路径
- `allow`：才真正放行

这意味着 Claude Code 从结构上否认了一种常见且危险的想法：模型懂了用户意图，就等于它有权代替用户执行。事实并非如此。理解意图不等于拥有授权，更不等于拥有持续授权。系统必须把“会做”和“可以做”分开。

从这个角度看，权限系统是在澄清代理角色。Claude Code 允许模型提出动作建议，但是否放行，由运行时、规则和用户决定。系统刻意把能力判断和授权判断分开。

4.5 权限结果本身也是一种运行时语义

在 `src/utils/permissions/PermissionResult.ts:23` 往后，系统甚至给权限行为准备了专门的描述函数：`allow`、`deny`、`ask`。这个细节很重要。它说明权限在 Claude Code 里不只是内部布尔值，而是有独立语义的运行时对象。

这件事之所以重要，是因为权限系统要让系统能够明确地表达“为什么这一步没有继续”。当一个代理说“我需要确认”时，系统是在声明责任边界。责任边界一旦说清楚，后续的拒绝、放行、缓存规则、临时授权、永久授权，才有地方安放。

更直接地说，一个代理系统如果连“这一步是我能做、不能做，还是需要问”的区别都说不清，就不该碰终端。因为终端不会替系统补完语义，终端只会执行。

4.6 StreamingToolExecutor 说明中断是一等语义

工具一旦开始并发和流式执行，中断问题就会立刻变得复杂。此时系统面对的是一个包含 queued、executing、completed、yielded 等多状态的队列，而不再只是单一动作。

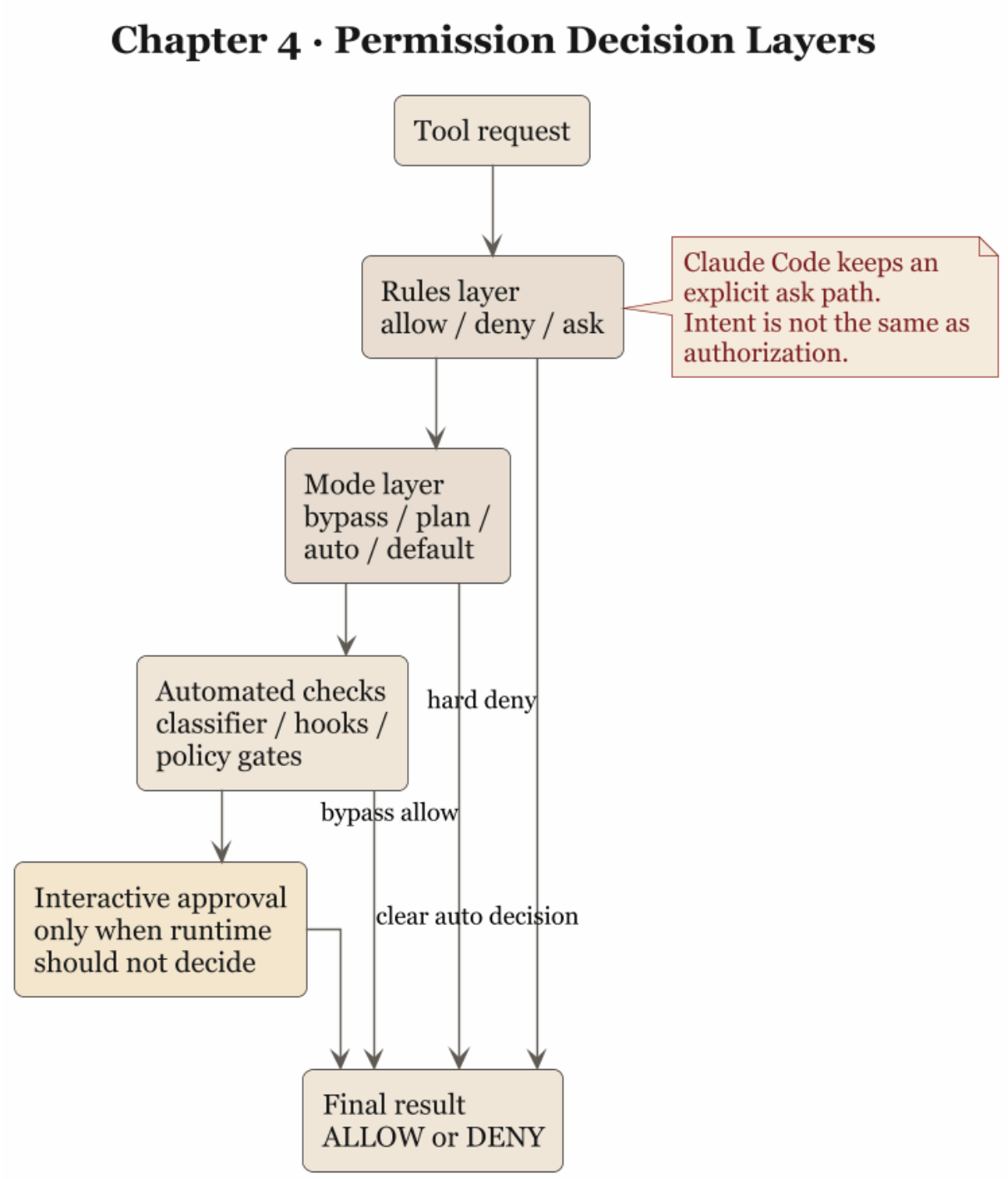

图 4: Claude Code Permission Decision Layers

Claude Code 在 `src/services/tools/StreamingToolExecutor.ts:34` 往后，明确把这套东西做成了一个独立的流式工具执行器。这里面最值得注意的是它如何处理中断和丢弃。

在 `StreamingToolExecutor.ts:64` 到 `:70`，系统允许在 streaming fallback 时整体 discard 当前工具集合；在 `:153` 到 `:205`，它会根据不同原因生成 synthetic error message，包括：

- sibling error
- user interrupted
- streaming fallback

到了 `:210` 往后，系统还会专门判断中断原因，区分：

- 因为别的并行工具出错而取消
- 因为用户 interrupt 而取消
- 因为 fallback 而放弃当前批次

更细一点，在 `:233` 往后，工具还有 `interruptBehavior`，决定它在用户插话时究竟该 `cancel` 还是 `block`。

这套设计的意义很大。它说明 Claude Code 并不把中断理解成“执行失败的一种特殊情况”，而是把中断当成和执行本身同样重要的语义。系统不仅要知道工具能不能开始，还要知道它被打断时如何收场、如何补齐结果、是否允许新消息插入。

这正是 Harness Engineering 的一个基本特点：不仅设计开始，也设计停下。没有停下语义的执行系统，最终只能依赖用户外部打断来补完设计。

4.7 Bash 为什么永远比别的工具更可疑

在 Claude Code 的工具世界里，Bash 不是普通工具，它更像风险放大器。原因很简单：它过于通用。越通用的接口，越难靠领域知识限制它。一个 file read tool 至少不会顺手杀进程，一个 grep tool 至少不会偷偷 push 代码，而 Bash 几乎什么都能做。

Claude Code 对 Bash 的不信任，写得相当实在。

一层是在 prompt 上，见 `src/tools/BashTool/prompt.ts:42` 往后。这里对 git、PR、危险命令、hook、force push、interactive flags 这些事情写了大量明确规则。那段 prompt 看上去啰嗦，实际上很有分寸：凡是后果大的地方，系统就不怕啰嗦。

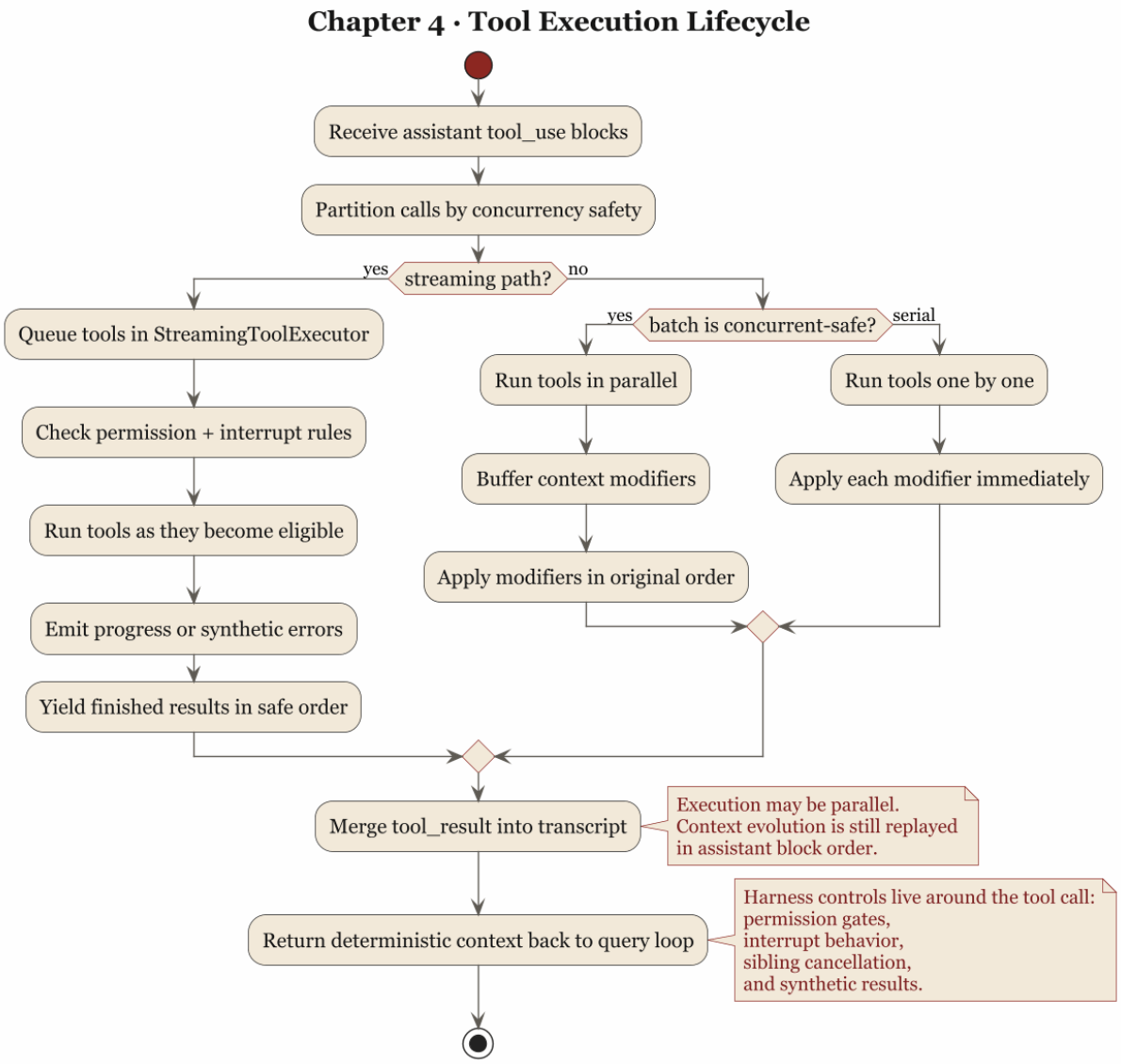

图 5: Claude Code Tool Execution Lifecycle

另一层是在权限和安全判定上。`src/tools/BashTool/bashPermissions.ts:1` 往后整整一大段，都在处理 shell 语义、命令前缀、重定向、wrapper、安全环境变量、classifier 与规则匹配。你从 `bashPermissions.ts:95` 往后甚至能看到系统为了防止复合命令导致检查失控，还专门给 subcommand 数量设了上限。

这说明，Bash 在 Claude Code 里一直被视为需要特殊审查的危险通道，不是普通的命令入口。工程师在这里承认了一件简单的事实：Bash 很强，所以必须被当成特例。

这是一个值得借鉴的判断。高风险能力不应该享受通用能力的待遇。能力越通用，越要特殊看管。把 Bash 当成普通工具，往往只是设计上的偷懒。

4.8 工具系统真正保护的，不只是用户，还包括系统自己

权限、调度和中断看起来像是在保护用户，其实它们同时也在保护系统自身。因为一个代理系统如果允许自己留下这些问题——不完整的 tool_result、失序的上下文修改、无边界的并发副作用、说不清楚的中断语义——最终最先崩掉的往往是系统的一致性。

这一点在 `query.ts` 里和工具执行层是互相咬合的。前一章提到，query loop 在中断时会补齐 synthetic tool_result；这一章看到，StreamingToolExecutor 也在内部预留了 discarded、hasErrored、siblingAbortController、interruptBehavior 等机制。两边一起作用，目的是让系统在“执行过什么、没执行完什么、为什么停了”这些问题上还能保持一条可追溯的因果链。

这也是 Harness 的核心含义之一：替系统保住秩序。很多约束表面上是在防止误操作，更深一层是在防止系统自己变成一堆无法解释的状态残片。

4.9 从源码里可以提炼出的第四个原则

这一章最后可以压成一句话：

> 工具是受管执行接口；权限是代理系统的基本器官。

Claude Code 的源码在几个地方共同支持这个判断：

- `toolOrchestration.ts` 把工具先分批，再执行，说明调度先于冲动
- `toolExecution.ts` 把 hooks、permission、telemetry 和 synthetic error 包在工具执行周围，说明执行不是裸调用

- `useCanUseTool.tsx` 把权限结果分成 `allow / deny / ask`，说明系统把授权当成独立语义
- `StreamingToolExecutor.ts` 为中断、fallback、并发出错预留专门语义，说明停止和开始同样重要
- `BashTool/prompt.ts` 与 `bashPermissions.ts` 对 Bash 采取特殊高压治理，说明高风险能力必须接受更密约束

如果要把这些提炼成可迁移的工程原则，大概可以写成这样几条：

- 让模型提出动作，不等于让模型拥有授权
- 工具调度必须保持因果秩序，哪怕执行并发
- 中断要有一等语义，不能靠异常兜底
- 高风险工具必须区别对待，不能图省事走通道化设计
- 一个工具系统真正保护的，既是用户，也是运行时本身

下一章要讨论的是这套系统里另一种常见错觉：上下文越多越好。Claude Code 的实现恰好说明，真正有经验的系统不会把上下文当仓库，而会把它当资源。接下来要讲的，是 memory、CLAUDE.md 与 compact 如何共同组成上下文治理。

第 5 章上下文治理：Memory、CLAUDE.md 与 Compact 是预算制度

5.1 上下文一多，系统就容易产生一种低级幻觉

人一旦可以往上下文里不停塞东西，就很容易相信一个朴素的神话：信息越多，系统越聪明。这个神话听起来甚至有点合情合理。毕竟知道得多，总比知道得少强。可惜代理系统不是图书馆，模型也不是藏书管理员。上下文不是一个“存进去就算拥有”的仓库，它首先是一笔昂贵、易膨胀、还会自我污染的预算。

Claude Code 的源码在这件事上很不浪漫。它并没有把上下文设计成一个可以无限堆叠的记忆池，反而在很多地方反复提醒自己：该加载什么、该截断什么、什么东西要长期保留、什么东西只能短期摘要，都是运行时必须严肃治理的事。

所以这一章要讨论的是：Claude Code 怎样防止自己被记住的东西拖死。这件事和“记住更多”看起来相近，工程上却是两种制度。前者偏向收藏癖，后者才接近治理术。

5.2 `CLAUDE.md` 体系说明，长期指令不能和临场对话混在一起

Claude Code 在 `src/utils/claudemd.ts` 开头就把记忆层次说得很清楚。它把 instruction source 分成几层：

- managed memory，例如 `/etc/claude-code/CLAUDE.md`
- user memory，例如 `~/.claude/CLAUDE.md`
- project memory，例如项目根目录里的 `CLAUDE.md`、`.claude/CLAUDE.md`、`.claude/rules/*.md`
- local memory，例如 `CLAUDE.local.md`

而且这些文件会按优先级和目录距离加载。离当前工作目录越近的 project 规则，优先级越高；越偏向私有、越偏向本地的规则，越晚加载，因而越靠近模型的注意力前沿。

这件事特别要紧。因为它说明 Claude Code 从一开始就拒绝把“长期协作规则”和“本轮临时对话”混成一锅粥。团队规范、个人偏好、仓库约束，这些东西的寿命远长于某一轮用户消息；如果把它们全都塞进聊天记录里，系统就会在两个极端之间摇摆：要么每轮都重复注入，浪费上下文；要么靠模型自己回忆，迟早失手。

`claudemd.ts` 给出的答案，是把这些稳定规则做成可发现、可分层、可组合的持久指令系统。还有个细节很有意思：它支持 `@include`，并且只允许一大批明确列出的文本扩展名。这说明工程师除了追求 include 的便利，也在提防另一种常见事故：有人把二进制、巨型文档、甚至不该进 prompt 的东西糊里糊涂带进来了。

这是正经工程师才有的克制。系统会先问：”什么东西值得进入系统记忆，什么东西一旦进入就是污染。”

5.3 `MEMORY.md` 是索引，不是日记本

如果 `CLAUDE.md` 管的是规则层，那么 `memdir` 处理的就是另一类更细的长期记忆。`src/memdir/memdir.ts` 里有一段设计很值得反复看：`ENTRYPOINT_NAME` 被定义成 `MEMORY.md`，但这个文件并不被鼓励用来直接堆内容，它被定义为 index。

源码里写得很实在。`buildMemoryLines()` 明确告诉模型，保存 memory 是两步：

1. 把具体 memory 写进独立文件
2. 再在 `MEMORY.md` 里加一个一行指针

为什么这么麻烦？因为系统知道入口文件天然会被频繁加载，而频繁加载的东西一旦变胖，整套上下文就会被它慢慢拖成一个不好收拾的胖子。

这也是为什么 `memdir.ts` 里专门有 `MAX_ENTRYPOINT_LINES = 200` 和 `MAX_ENTRYPOINT_BYTES = 25_000`。超过了，系统会直接 `truncateEntrypointContent()`，并在结尾追加明确警告：只加载了一部分，请把细节移到 topic files。

这套做法特别像一个见过太多失控索引的人。它不相信大家会天然克制，所以把“入口必须短”做成硬约束。因为入口文件一旦既当目录又当正文，最后就既不是目录，也不是正文，只是一个谁都不愿再读第二遍的烂尾摘要。

从 Harness Engineering 的角度看，这里抽出来的原则非常清楚：长期记忆必须分成“入口”和“正文”。入口负责低成本寻址，正文负责高密度承载。把两者混为一谈，最终一定是入口失效，随后整套记忆系统退化成摆设。

5.4 Session memory 说明，短期连续性也不能靠聊天记录硬扛

只有长期 memory 还不够。代理系统真正难受的地方，常常在于“这轮之前我们到底做到哪一步了”。这是一次会话内部的连续性问题。

Claude Code 在 `src/services/SessionMemory/prompts.ts` 里专门给这件事建了一套模板。默认模板里有这些栏目：

- `Current State`
- `Task specification`
- `Files and Functions`
- `Workflow`
- `Errors & Corrections`
- `Codebase and System Documentation`
- `Learnings`
- `Key results`
- `Worklog`

你一看就知道，这不是给人抒情的。它关心的是：现在做到哪了，踩过什么坑，改过哪些文件，后面该接什么。更有意思的是更新 prompt 的语气。源码里明确要求：

- 只能用 Edit tool 更新 notes file
- 不要提 note-taking 这件事本身
- 不要改模板结构
- `Current State` 必须始终反映最近工作
- 每节都要信息密集，但要控制预算

这说明 session memory 在 Claude Code 里并非“另存一份聊天记录”，它会把当前会话萃取成一种可继续工作的操作说明书。它不求完整复刻对话，而求压缩出未来继续干活所必需的骨架。

这里有个极其工程化的细节。`prompts.ts` 里定义了 `MAX_SECTION_LENGTH = 2000` 和 `MAX_TOTAL_SESSION_MEMORY_TOKENS = 12000`。超过预算，系统不会夸你记得细，而是要求你 aggressively condense，尤其优先保留 `Current State` 和 `Errors & Corrections`。

这很能说明问题。真正成熟的系统，会把“为继续工作保留最有用的部分”当成美德。因为上下文预算是工作内存。工作内存的第一职责是可操作。

5.5 自动 compact 说明，上下文治理首先是预算治理

到这里，长期规则、持久 memory、session memory 都有了，但上下文还是会膨胀。于是 Claude Code 在 `src/services/compact/autoCompact.ts` 里进一步承认一个现实：不管你多会整理，只要对话够长，总会逼近窗口边缘。

`getEffectiveContextWindowSize()` 先把模型 context window 减去一笔保留给 summary 输出的预算。`MAX_OUTPUT_TOKENS_FOR_SUMMARY` 直接预留了 20,000 tokens。也就是说，系统先假定 compact 本身要花钱，绝不把窗口吃到只剩一口气时才想起求生。

接着 `getAutoCompactThreshold()` 又在有效窗口上再扣掉 `AUTOCOMPACT_BUFFER_TOKENS = 13_000`。警告阈值、错误阈值、手动 compact 预留空间，也都各自分出 buffer。

这套数字背后有个很朴素的道理：上下文治理需要提前为失败和恢复留出余地。不留余地的系统，平时看着像节俭，出事时才暴露真相——不过是把风险账单留给了下一轮。

更有意思的是 `AutoCompactTrackingState`。它不仅记 `compacted`，还记 `turnCounter`、`turnId` 和 `consecutiveFailures`。这说明 autocompact 是一段会被追踪、会失败、会被限流的运行时行为。

源码甚至写了一个很直白的注释：全球每天曾经浪费大量 API calls 在连续失败的 autocompact 上，所以 `MAX_CONSECUTIVE_AUTOCOMPACT_FAILURES = 3`，再失败就触发 circuit breaker。这里的气质非常好，像一个终于受不了浪费的人：你可以失败，但不能无限次、无记忆地失败。

5.6 `compactConversation()` 说明，摘要要重建可继续工作的上下文

很多人一听 compact，会以为就是“把前面聊天摘要一下”。Claude Code 的实现要复杂得多。`src/services/compact/compact.ts` 里的 `compactConversation()` 真正做的，是把原有上下文拆开、摘要、再注入必要附件，重新搭出一个还能工作的后 compact 世界。

先看压缩前的清洗。`stripImagesFromMessages()` 会把图片、文档替成 `[image]`、`[document]` 之类的标记；`stripReinjectedAttachments()` 会把反正之后还要重新注入的 attachment 先剥掉，免得浪费 token。仅这两个动作就说明，compact 会有选择地丢掉那些”对摘要没用、但 token 开销极大”的部分。

再看摘要失败时的处理。源码里有 `truncateHeadForPTLRetry()`,专门应对“compact 请求自己都 prompt too long”的尴尬场面。也就是说，系统不仅承认主流程会爆，还承认“救火工具本身也会爆”。这很像真实世界，而不是 demo。

而在 compact 成功之后，Claude Code 做的不是简单保留一条 summary。它还会：

- 清空旧的 `readFileState`
- 重新生成 post-compact file attachments
- 把 plan attachment 补回来
- 把 plan mode attachment 补回来
- 把 invoked skills attachment 补回来
- 把 deferred tools、agent listing、MCP instructions 的 delta attachment 重新补回来
- 执行 session start hooks 和 post-compact hooks
- 写 compact boundary message，记录 pre-compact token 数与边界信息

这些动作合在一起，意思很明确：compact 的目标是把“继续干活所需的运行时环境”重新铺平。摘要只是中间产物，不是最终目的。

所以 compact 在 Claude Code 里更像一次受控重启，而不是一次聊天总结。旧上下文会被转译成新的工作底座。这种设计很值得记住，因为很多系统只做前半截，结果 compact 之后虽然“还记得大概”，却已经失去了工具状态、计划状态、附件状态，接下来还得再花几轮找回自己。

5.7 上下文治理的关键是保留工作语义

如果只看 `compact.ts` 的后半段，会发现一个贯穿始终的倾向：Claude Code 真正在意的是把工作语义保住。

例如它会恢复最近访问文件的 attachment，因为这些文件往往构成当前工作面的局部现实；它会恢复 plan mode，因为否则模型压缩完以后可能忘了自己还处在 plan discipline 里；它会保留 invoked skills 的内容，但又给每个 skill 设置 token cap，避免 skill 本身在 post-compact 阶段反客为主。

源码里这句话很有味道：per-skill truncation beats dropping。意思是，即使要裁，也优先保住开头那一段最关键指令，而不是整个扔掉。这就是治理，不是纯粹节流。纯节流是砍，治理是知道该砍哪里、该保什么。

从这里可以抽出一个相当稳妥的经验：上下文系统应该优先保留能维持行动语义的东西，而不是优先保留看起来信息量最大的东西。文件细节、当前计划、错误修正、技能

约束，这些都直接决定下一步能不能做对。反过来，冗长的历史对话、重复出现的附件、运行时随时可以重新拿到的东西，就没必要再占着座位。

5.8 从源码里可以提炼出的第五个原则

这一章最后可以压成一句话：

> 上下文是工作内存。治理它的目标是支持系统继续工作。

Claude Code 的源码在几个层面共同支持这个判断：

- `claudemd.ts` 把长期指令分层加载，说明稳定规则要和临时对话分开治理
- `memdir.ts` 把 `MEMORY.md` 定义成索引并强行截断，说明入口文件必须短而可寻址
- `SessionMemory/prompts.ts` 用固定模板提炼会话连续性，并对 section 和总量设预算，说明短期记忆也必须结构化
- `autoCompact.ts` 为 compact 预留输出预算、缓冲区和失败熔断，说明上下文窗口要按风险来经营
- `compact.ts` 在摘要后恢复计划、文件、技能、工具附件和 hook 状态，说明 compact 的目标是重建工作语义，而不是写一段好看的总结

如果把这些抽象成可迁移的工程原则，大概有这样几条：

- 长期规则、长期记忆、会话连续性，应该分层，不该混写
- 入口型记忆必须短小，否则整个系统会被入口拖垮
- session summary 应该服务于“继续工作”，而不是服务于“回忆完整”
- compact 是上下文治理主路径
- 压缩后的上下文必须保住运行语义，而不是只保住语言表面

下一章要讲的是这套治理系统碰到极限时怎么办。因为一个真系统终究会出错：prompt too long，max output tokens，hook 死循环，恢复分支相互打架。到那时你才看得出来，一个代理系统到底是在“赌不出事”，还是在认真设计出事之后如何继续运行。

第 6 章错误与恢复：出错后仍能继续工作的代理系统

6.1 工程世界最不值得相信的话，就是“正常情况下”

很多系统设计文档里，最常见的偷懒方式，就是先讲一遍“正常情况下”的流程，仿佛只要主路径足够漂亮，错误就会自动显得次要。可代理系统一旦进入真实运行环境，这种写法通常很快就会露馅。因为现实中什么都会出问题：

- 模型会被截断
- 请求会超长
- hook 会制造回环
- 工具会中断
- fallback 会发生
- 恢复逻辑本身也会失手

所以，判断一个代理系统成熟不成熟，不能只看它回答顺畅的时候有多像个人，而要看它出故障的时候像不像系统。前者容易靠一点 prompt 工程粉饰，后者只能靠运行时纪律。

Claude Code 在这一点上的可取之处，是它没有假装自己不会出错。相反，源码里反复体现出一种冷静判断：错误属于主路径，恢复则是必须提前设计好的运行机制。

6.2 `prompt too long` 是一种必然周期

对长会话代理来说，`prompt too long` 是一种迟早会来的季节变化。你如果把它当偶发异常，系统迟早会被它教育。

Claude Code 的 `query.ts` 就没有把它当偶发异常处理。在 `src/query.ts` 里，系统甚至会“暂时扣下”这类错误，不立刻把它原样抛给用户。流式阶段里，`withheld` 逻辑会识别 recoverable errors，包括：

- prompt too long
- media size error
- max output tokens

这件事的意思很明确：有些错误要先交给恢复系统试着处理，再决定是否展示给用户。这个顺序很关键，因为用户真正关心的通常是系统还能不能继续干活。

在 prompt too long 的分支上，Claude Code 先尝试更便宜、更保守的恢复路径。若启用了 context collapse，就先 `recoverFromOverflow()`，把已经 staged 的 collapse 提交掉；如果还不行，再进入 `reactiveCompact.tryReactiveCompact()`。也就是说，恢复是分层的：先排空已知积压，再做更重的全文压缩，不会一上来就重建世界。

这种次序特别有工程味。因为真正好的恢复系统，不会把所有错误都交给“最重的一把锤子”。它会先试图保住最细粒度的上下文，再在必要时接受更粗糙的摘要替代。

6.3 响应式 compact 说明，恢复的关键在于别把自己逼进死循环

很多系统做恢复时容易犯一个愚蠢但常见的错误：一旦发现错误可恢复，就不停重试，直到把错误从偶发事件升级成资源灾难。

Claude Code 对这件事非常警惕。`query.ts` 里有两个地方都能看出这种警惕。

第一处，是 `hasAttemptedReactiveCompact`。一旦 reactive compact 已经试过，再次遇到同类问题时，系统不会装傻重来。因为工程师很明白：如果 compact 之后还是不行，那么继续 compact 大概率只是在把同一种失败换个姿势再演一次。

第二处，是 stop hooks 的防死循环处理。源码里有非常直白的注释：如果 prompt too long 之后还让 stop hooks 介入，就可能出现 death spiral，路径大致是：

错误 -> hook blocking -> retry -> 错误 -> hook blocking

这话写得一点也不文学，却比许多文学都诚实。因为它承认系统里最危险的错误，是失败分支和恢复分支彼此咬住，开始无限自我复制。

所以 Claude Code 在 prompt too long 无法恢复时，会直接 surface 错误并跳过 stop hooks。原因很简单：这时候继续走形式流程，只会让坏事更有仪式感。

6.4 max_output_tokens 的处理说明，恢复要以续写为主

大模型产品有个很坏的习惯，一旦输出截断，就先来一段很客气的废话：抱歉，刚才被截断了，我来总结一下。听上去态度不错，实际上对工作几乎没帮助。

Claude Code 在 `src/query.ts:1185` 往后的处理，明显更接近工程系统。它先试一种成本较低的恢复：如果当前使用的是较保守 cap，就把 `maxOutputTokensOverride` 提升到更高值，直接重跑同一请求。注意，这一步没有插入 meta message，也没有让模型先寒暄，系统先给它一次把原任务做完的机会。

如果更高 cap 也不够，再进入第二层恢复：给模型追加一条 meta user message，内容非常实在，大意是：

直接继续，不要道歉，不要 recap，若中断发生在半句，就从半句接着写；剩余工作拆小一点。

这是一条很有启发意义的系统指令。它说明 Claude Code 对恢复的理解，是尽量保持任务连续性，不要把额外 token 花在礼貌性收尾上。在长任务里，这种区别非常大。因为每一次截断后的 recap，都会进一步消耗预算，并且增加语义漂移。最后系统做的就不再是任务本身，而是一轮轮地回顾自己做任务。

所以对 `max_output_tokens` 来说，较好的恢复通常是续写。Claude Code 在这里优先保证任务连续性，而不是补充礼貌性说明。

6.5 auto compact 的失败熔断，说明恢复系统自己也要受治理

如果说前面讲的是“单次错误怎么救”，那 `src/services/compact/autoCompact.ts` 处理的就是另一个层面的问题：当恢复机制本身不断失败时，怎么办。

源码的回答很简单，也很正确：别一直试。

`AutoCompactTrackingState` 里专门有 `consecutiveFailures`。一旦失败次数超过阈值，`shouldAutoCompact` 即便判断“按理说该 compact 了”，系统也会直接跳过。源码注释甚至给出过往数据：曾有大量 session 在连续 autocompact failure 上白白烧掉海量 API calls，所以必须加 circuit breaker。

失败熔断的本质，是承认当前恢复手段在这个局面里已经失效。一个真正成熟的系统，不能只会在成功时记录指标，也得在失败时懂得收手。不会收手的恢复系统，和不会刹车的汽车差不多，理论上都叫系统，实际上都不该上路。

从 Harness Engineering 角度看，这里可以抽出一条很硬的原则：任何自动恢复机制都必须可计数、可限次、可熔断。否则恢复会从保险丝变成新的起火点。

6.6 compact 自己也会爆，所以连“修复动作”都需要修复策略

`compactConversation()` 这段代码还有一个很动人的现实主义时刻：它承认 compact 请求自己也可能 `prompt too long`。

这件事看上去带一点黑色幽默。系统为了缩短上下文去发摘要请求，结果摘要请求也因为上下文太长而失败。很多人不喜欢这种情形，因为它暴露得过于直接。但工程系统首先要解决的是继续运行，而不是保持表面完整。

Claude Code 的处理方式，是在 `compact.ts` 里引入 `truncateHeadForPTLRetry()`。当 compact 自己太长时，系统会先把更早的 API round 成组地从头部剥掉，再重试 compact，避免让用户卡在“连压缩都压不动”的状态里。

这里的取舍很清楚：这种修复有损，也会丢历史，但它优先保证用户不被完全锁死。源码注释写得很实在，这是一种 last-resort escape hatch。

这种处理方式的价值，在于它没有回避现实约束。系统快要窒息时，优先级是先恢复呼吸，再讨论信息保真度。这个判断不追求漂亮，但很实用。

6.7 abort 语义说明，中断也属于错误恢复的一部分

很多人把 abort 单独归到交互体验，不愿意把它放进错误恢复讨论里。从运行时角度看，中断就是一种必须被正确回收的失败态。

Claude Code 在两个层面都认真处理了这件事。

一层在 `query.ts`。如果 streaming 时用户打断，系统会先消费 `StreamingToolExecutor.getRemainingResults()`，为已经发出但尚未完成的工具生成 synthetic tool_result，确保前面承诺过的 `tool_use` 不会变成悬空债务。

另一层在 `compact.ts`。源码里专门把 compact 的 abort controller 传给 forked agent，并处理 `APIUserAbortError`，防止“被用户按了 Esc 的 compact”误算成一次成功摘要。

这两处连在一起看，意思很明确：中断不只是“用户不想看了”，而是一次需要正确收尾的状态转移。错误恢复如果只管异常、不管中断，最终会留下大量语义半残的执行轨

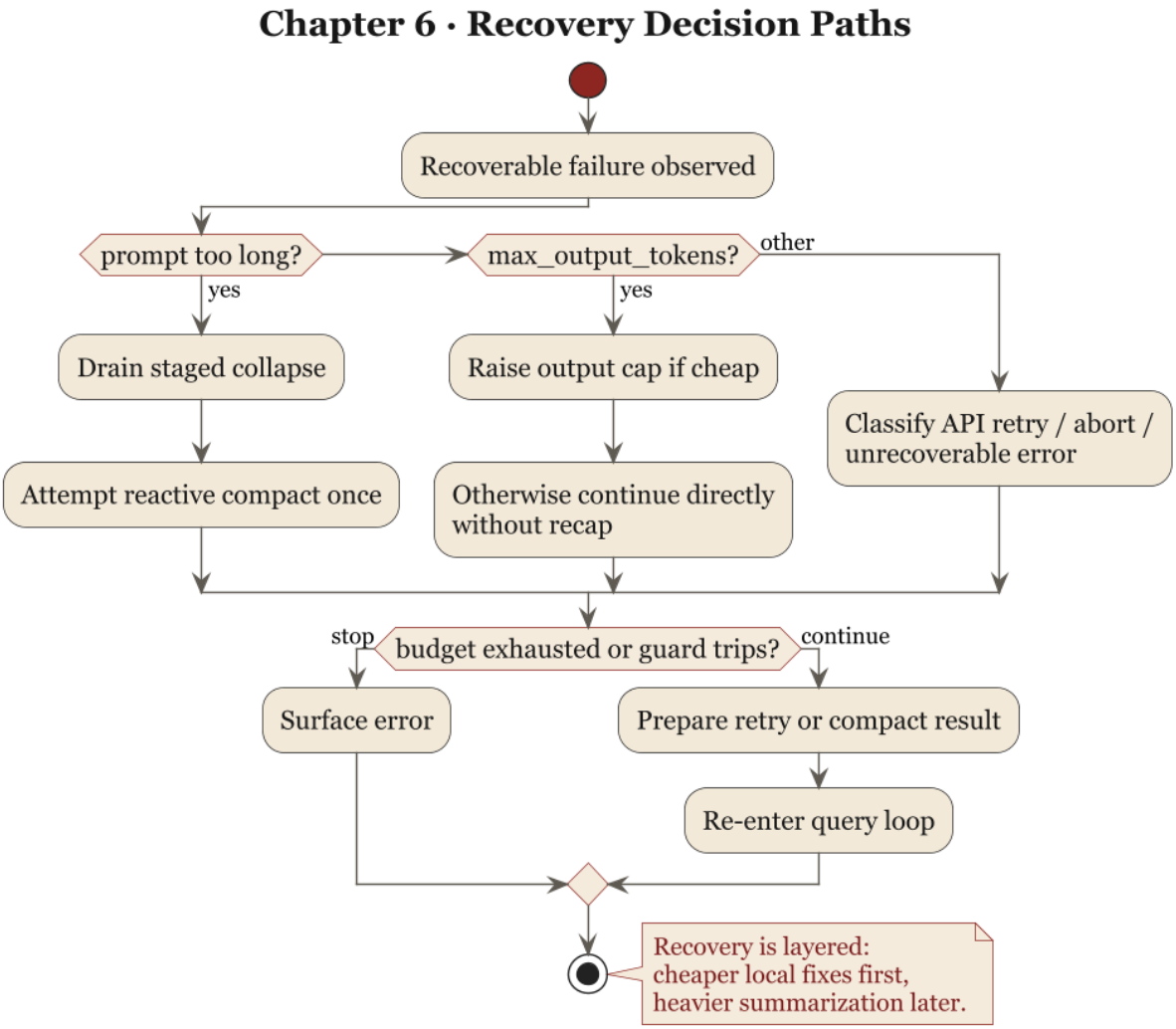

图 6: Claude Code Recovery Decision Paths

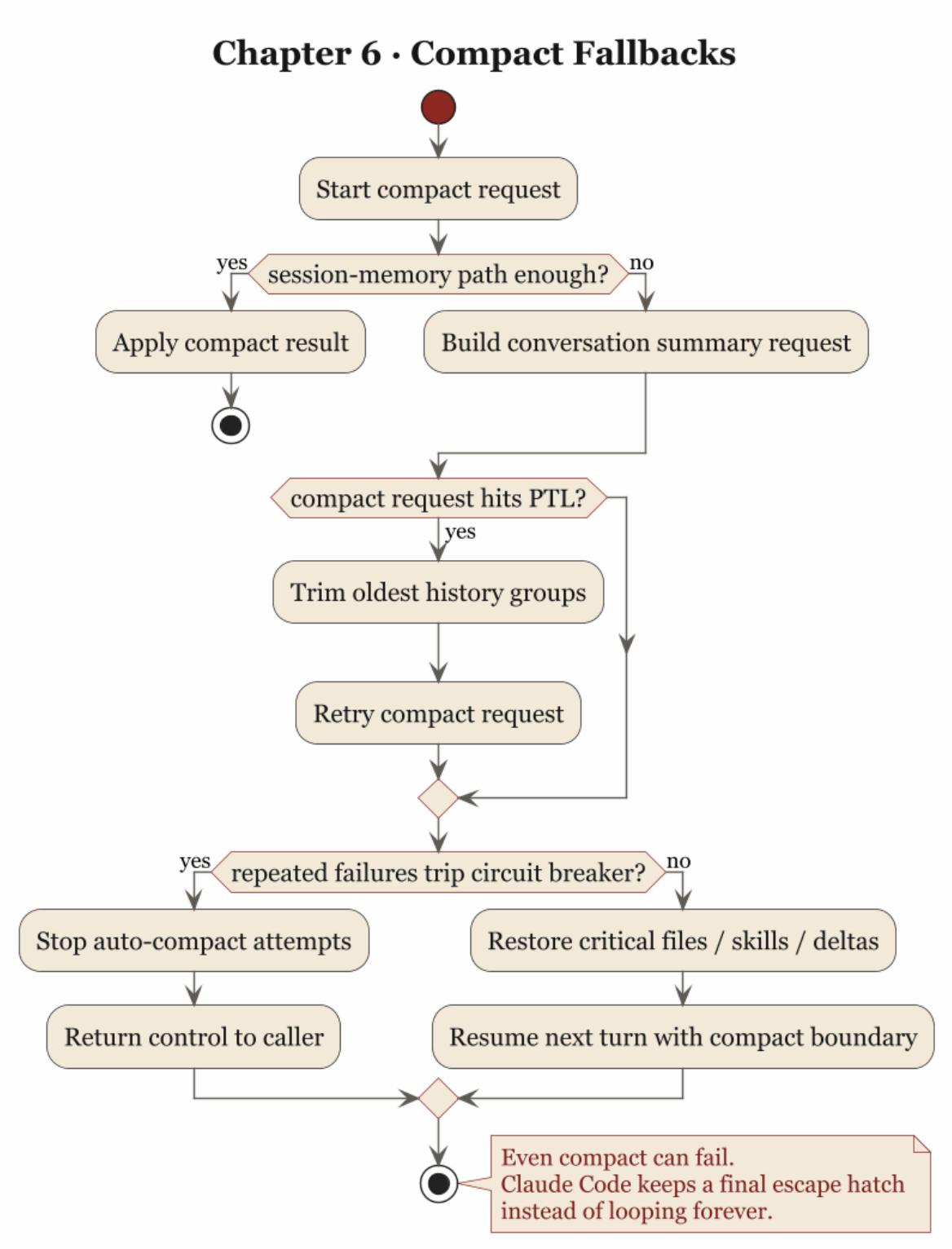

图 7: Claude Code Compact Fallbacks

迹。那种轨迹通常短期内没人查，长期看全是祸根。

6.8 错误处理真正保护的，是执行叙事的一致性

把这些源码放在一起看，会发现 Claude Code 对错误与恢复的理解，有一个很核心但常被忽略的目标：保护执行叙事的一致性。

什么叫执行叙事？很简单，就是系统还能不能说清楚：

- 我刚才试图做什么
- 为什么没做成
- 我用了什么恢复路径
- 现在是继续、停止，还是换轨

`query.ts` 里的 `transition.reason`，`maxOutputTokensRecoveryCount`，`hasAttemptedReactiveCompact`，以及 compact boundary、synthetic error message 这些东西，都是为了让这条叙事线不断裂。它们是为了让系统自己别失忆。

一个没有叙事一致性的代理系统，表面也许还能继续输出，但它的内部已经开始散了：

- 今天，用户看到的是多说几句废话
- 明天，运维看到的是一会儿 hook retry、一会儿 compact retry，理不清因果
- 后天，团队看到的是系统出了问题，谁都说不清它到底经历了什么

所以错误恢复真正修补的，不只是错误本身，还有系统对自己行为的解释能力。解释能力一断，系统就会从工程对象退化成玄学对象。

6.9 从源码里可以提炼出的第六个原则

这一章最后可以收成一句话：

> 代理系统是否可靠，体现在错误发生后仍能维持可解释、可限界、可继续的执行秩序。

Claude Code 的源码在几个点上共同支持这个判断：

- `query.ts` 会暂时扣下可恢复错误，先交给恢复分支处理，说明错误要先尝试转化

- prompt-too-long 恢复先走 collapse drain，再走 reactive compact，说明恢复路径按成本和破坏性分层
- `hasAttemptedReactiveCompact` 与 stop hook guard 明确防止死循环，说明恢复本身也要受治理
- `max_output_tokens` 先提 cap，再要求模型直接续写，说明恢复的目标是延续任务，不是补充礼貌动作
- `autoCompact.ts` 的 consecutive failure 与 circuit breaker，说明自动恢复必须可熔断
- `compact.ts` 对 compaction 自身的 prompt-too-long 也有降级修复，说明连修复动作本身都要有恢复策略

如果把这些抽成可迁移的工程原则，大概是这样：

- 错误恢复要分层，不要所有问题都打一把重锤
- 恢复逻辑必须防止自我回环
- 自动恢复需要计数和熔断
- 截断后的最佳恢复通常是续写，不是总结
- 中断也是一种需要语义收尾的失败态
- 一个系统是否可靠，最终要看它出错后还能不能把自己的行为讲明白

下一章要进入另一类更棘手的问题：多代理与验证。因为当一个系统不再只是“自己出错自己救”，而开始把任务分给别的 agent，再把结果收回来复核，错误与恢复的问题就从单线程秩序升级成了组织问题。那时你面对的不只是一个模型会不会失手，而是一群不稳定执行体如何彼此约束。

第 7 章多代理与验证：用分工和验证管理不稳定性

7.1 单代理走到一定程度，问题就不再是“会不会做”，而是“怎么分工”

一个代理如果只在单线程里回答问题，很多矛盾都还能靠耐心遮过去。它慢一点，用户多等会儿；它想得乱一点，多追问几轮；它偶尔把上下文拖成一团，也还可以靠 compact 补救。可一旦任务变大，单代理模型就会碰到一个更难缠的问题：研究、实现、验证都挤在同一条上下文链上，彼此抢预算、抢注意力、抢叙事中心。

这时候，多代理看上去像一种自然答案。再开几个 worker，不就行了？但事情没这么便宜。多代理并不天然带来秩序，很多时候它只会把单代理的混乱并行复制几份。真正困难的是隔离这些 agent 的不稳定性，同时把结果组织回来。

Claude Code 的源码在这点上很清醒。它没有把 subagent 当成“另一个会说话的窗口”，而是把它当成一段需要明确缓存边界、状态边界、验证职责和清理责任的受管执行流程。

7.2 forked agent 的第一原则是 cache-safe

`src/utils/forkedAgent.ts` 开头有一段注释，非常能说明 Claude Code 对 subagent 的真实理解。它说 forked agent utility 的职责包括：

1. 与父代理共享 cache-critical params，确保 prompt cache hit
2. 跟踪整个 query loop 的 usage
3. 记录指标
4. 隔离可变状态，防止干扰主循环

这四条里，最先出现的是“共享 cache-critical params”。这并非偶然。它说明在 Claude Code 眼里，fork 是运行时层面的受控分叉。既然是分叉，就必须非常在意哪些参数必须和父请求保持一致，否则 prompt cache 共享就失效，成本和延迟会立刻变坏。

`CacheSafeParams` 里明明白白列了这些要素：

- `systemPrompt`
- `userContext`
- `systemContext`
- `toolUseContext`
- `forkContextMessages`

还专门提醒：别随便改 `maxOutputTokens`，因为 thinking config 也会受影响，而 thinking config 又是 cache key 的一部分。

这段设计说明，多代理首先是运行时经济学问题。一个子代理如果每次都把父上下文重新烧一遍 token，看上去像在并行提效，实际只是把浪费并行化。Claude Code 在这个环节先处理的是：怎么 fork 才不把缓存打烂。

7.3 状态隔离说明，子代理首先要减少污染

forked agent 的第二个关键，在 `createSubagentContext()`。源码里对它的默认行为写得很直白：默认情况下，所有 mutable state 都隔离，避免干扰 parent。

它默认会做这些事：

- `readFileState` 先 clone
- `abortController` 生成 child controller，而不是直接共享
- `getAppState` 做包装，让子代理避免 permission prompt
- `setAppState` 默认 no-op
- `nestedMemoryAttachmentTriggers`、`loadedNestedMemoryPaths` 等集合都重新建

只有在明确 opt-in 的情况下，才会共享某些 callback，例如 `shareSetAppState`、`shareSetResponseLength`、`shareAbortController`。

这套设计特别重要，因为它揭示了一个很多人做多代理时都会忽略的事实：子代理最宝贵的地方，在于它可以避免把自己的局部混乱污染主线程。研究中的误判、临时读到的

文件状态、一次性的推理枝杈、正在进行的工具决策，如果全都直接写回主上下文，你得到的只会是更快的脏化。

Claude Code 在这里的态度是：共享要靠明确同意，隔离才是默认伦理。这种伦理很像数据库事务设计，不像聊天玩具。它不假定“大家都是自己人，状态可以随便串”，而是假定“只要是可变状态，就必须先隔离，再决定共享哪些部分”。

7.4 协调者模式说明，synthesis 才是稀缺能力

如果只看 `src/coordinator/coordinatorMode.ts`，你会发现 Claude Code 对 coordinator 的要求很有分寸。它明确说 coordinator 的工作包括：

- 帮用户达成目标
- 指挥 worker 做 research、implementation、verification
- 综合结果并和用户沟通
- 能直接回答的问题就直接回答，不要滥委派

最关键的一句，在第 5 节 prompt 里：**Always synthesize**。当 worker 回报研究结果后，协调者必须先读懂，再写出具体 prompt；不要说“based on your findings”，不要把理解继续外包给 worker。

这句话几乎就是多代理系统的命门。因为真正稀缺的是有人把 worker 带回来的局部知识重新压成清晰、可执行、可验证的下一步。缺少这一层，多代理很快就会退化成一种带着礼貌措辞的任务转发机。每个 agent 都在忙，系统整体却并没有更懂。

Claude Code 至少在 prompt 设计上很明白这个道理。它要求 research 和 synthesis 分开，要求协调者对研究结果负责。后续 prompt 里必须出现具体文件、具体位置、具体变更，而不是抽象地”根据前面的结论”。这是非常正统的工程分工：研究可以分布式，但理解必须重新收束。

7.5 验证必须独立成阶段，否则“实现完成”很快就会冒充“问题解决”

`coordinatorMode.ts` 还有一段特别值得抄下来。它把常见任务分成：

- Research
- Synthesis

- Implementation
- Verification

并且专门强调：verification 的目标是证明代码有效，而不只是确认代码存在。源码里甚至写得近乎不留情面：

- run tests with the feature enabled
- investigate errors, don’t dismiss as unrelated
- be skeptical
- test independently, don’t rubber-stamp

这段话说明 Claude Code 没把验证当成实现 worker 顺手带一下的附属环节，而是当成第二层质量关。你甚至能在 prompt 里看到“implementation worker 自证一遍，verification worker 再作为第二层 QA”这种明确分层。

为什么这点这么重要？因为在代理系统里，“我改了代码”和“代码因此正确”之间，隔着一条很宽的河。模型尤其擅长在这条河上搭纸桥。它会给你改动、解释、甚至给你一段像样的测试输出，但这些都不等于功能真的在系统里站住了。

所以，把 verification 单列出来，是为了防止“会改代码”冒充“能交付结果”。Harness Engineering 在多代理阶段真正需要的，正是这种角色分化。实现的人要尽量专注于改；验证的人要专门怀疑这些改动配不配活着。

7.6 hooks 和任务生命周期说明，子代理不是扔出去就算了

多代理系统还有一个很容易被忽略的地方：spawn 只是开头，收尾同样重要。

`src/utils/hooks/hooksConfigManager.ts` 里定义了 `SubagentStart` 和 `SubagentStop` 两类 hook。前者在 subagent 启动时触发，输入里有 `agent_id` 和 `agent_type`；后者在 subagent 即将结束时触发，输入里还带 `agent_transcript_path`，并允许 exit code 2 把 stderr 反馈给 subagent，继续让它跑。

这说明子代理在 Claude Code 里是显式暴露生命周期节点的系统对象。启动时可以观测，停止前可以介入，转录路径可追踪。这里的重点在于，“子代理结束”也是需要被管理的事件。

与此同时，`src/tasks/LocalAgentTask/LocalAgentTask.tsx` 的 `registerAsyncAgent()` 又展示了另一个层面：每个 async agent 都会注册 cleanup handler，父 abort 可以自动传播给子 abort controller。任务结束后还要 evict output、更新状态、解除 cleanup 注册。

这套机制非常像操作系统，不像聊天面板。它关心的核心问题是：

- 这个 agent 是否仍在运行
- 父任务死了它是否该跟着死
- 它的输出文件是否还要保留
- 它的 cleanup callback 有没有泄漏

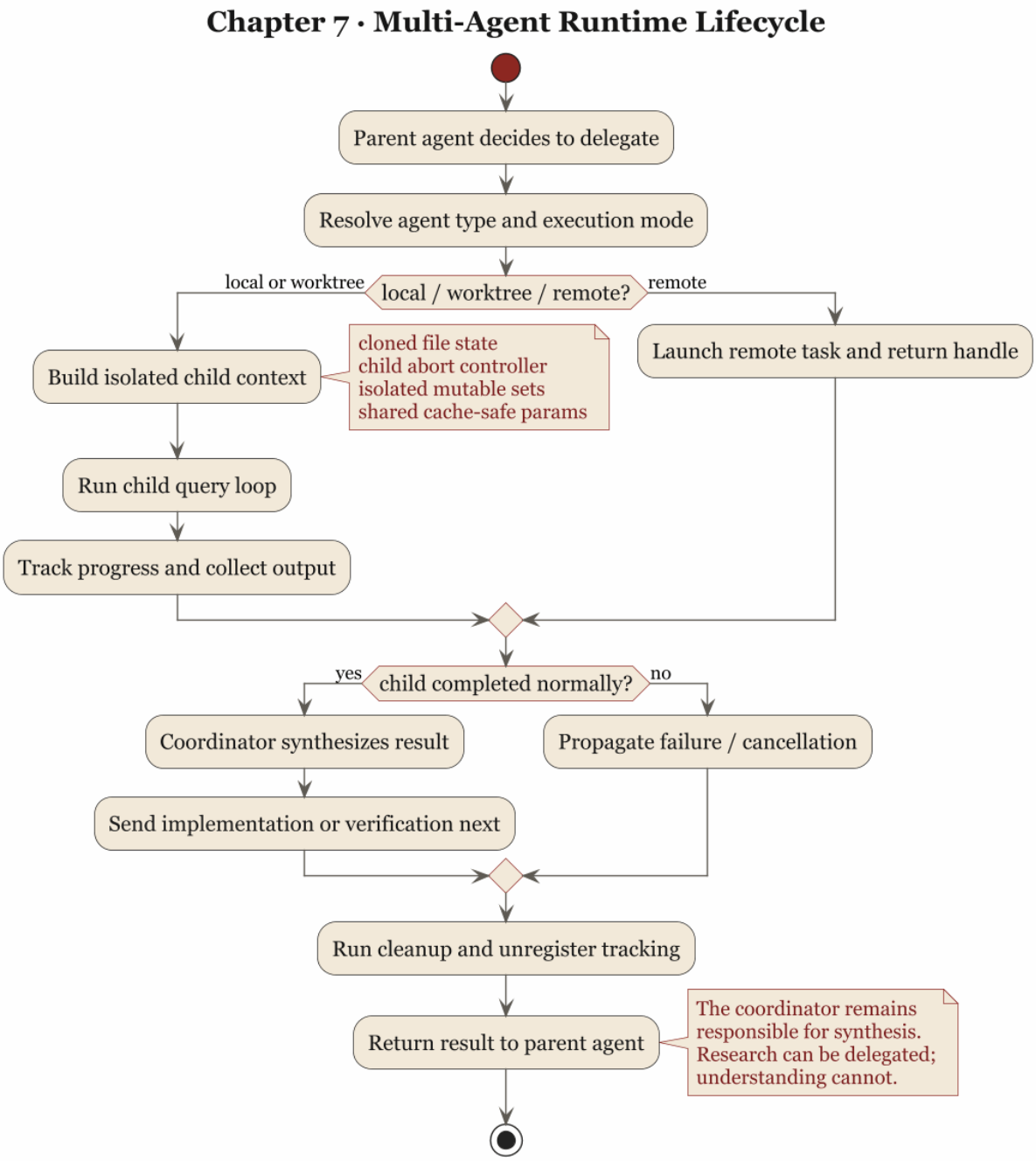

图 8: Claude Code Multi-Agent Runtime Lifecycle

很多多代理 demo 都只做到“我能再起一个 agent”，Claude Code 至少多做了一步：

它把 agent 当作会泄漏资源、会残留状态、会在父进程结束后变成孤儿的运行实体来看待。这才像是在把代理当系统组件处理。

7.7 验证不仅针对代码，也针对记忆和建议

多代理与验证并不只发生在 code change 之后。Claude Code 在 memory 体系里也埋了一条很值得注意的原则。

`src/memdir/memoryTypes.ts` 里专门提醒：memory records can become stale；在基于 memory 给用户建议之前，要先 verify current state；如果记忆与现状冲突，要相信眼下读到的真实状态，并更新或删除 stale memory。

这句话放在多代理章节里，恰好能说明一个更一般的事实：验证是整个系统用来抵抗时间漂移和上下文漂移的基本习惯。一个系统如果只验证新写下去的代码，却不验证旧记忆、旧假设、旧索引，那它仍然会被历史信息带偏。

从这个角度看，verify 既是一项 skill，也是一种组织纪律。你可以把工作分出去，可以把信息存起来，可以让其他 agent 先跑在前面，但在用户准备据此行动之前，总要有人回到当前现实，重新确认这些东西还是真的。

7.8 多代理真正解决的是不确定性的分区

把这些源码拼起来看，会发现 Claude Code 的多代理设计其实围绕一个朴素目标展开：给不确定性分区。

研究 worker 可以在局部上下文里探索，不必把所有试探都写回主线程。实现 worker 可以专注修改，不必同时扛着全局沟通负担。验证 worker 可以专门怀疑，不必替自己的实现辩护。coordinator 则留在中间做收束、综合和用户界面。

这套分区带来的最大好处是职责清晰。职责一清晰，错误就更容易定位：

- 是 research 没找到关键线索
- 还是 synthesis 没吃透研究
- 还是 implementation 写错了
- 还是 verification 放水了

反过来，如果所有事情都交给一个代理顺手完成，你最后得到的只是一锅浓汤。味道也许不错，出了问题却没法分层修。

所以，多代理真正有价值的地方，在于把不同种类的不确定性关进不同容器里，再用 coordinator 把它们组织回来。这种做法比单纯追求并发更稳，也更符合工程要求。

7.9 从源码里可以提炼出的第七个原则

这一章最后可以压成一句话：

> 多代理依赖清晰分工：研究、实现、验证和综合各自处在不同约束容器里，最后由协调者把结果重新缝合成可交付结果。

Claude Code 的源码在几个地方共同支持这个判断：

- `forkedAgent.ts` 把 cache-safe 参数、usage tracking 与状态隔离放在第一位，说明 fork 首先是运行时控制问题
- `createSubagentContext()` 默认隔离 mutable state，只允许显式 opt-in 共享，说明多代理先防污染再谈协作
- `coordinatorMode.ts` 强调 coordinator 必须 synthesize，而不是转发研究结果，说明综合理解不能外包
- 同一个文件把 verification 独立成阶段，并要求独立证明变更有效，说明实现与验证必须角色分离
- `hooksConfigManager.ts` 提供 `SubagentStart` / `SubagentStop` 生命周期 hook，说明子代理是可观测对象，不是黑箱线程
- `LocalAgentTask.tsx` 处理 parent abort、cleanup、output eviction，说明 agent 生命周期需要回收机制

如果把这些提炼成可迁移的工程原则，大概是这样：

- fork 时先考虑 cache 和状态边界，再考虑“人格分工”
- 子代理默认应隔离，可共享必须显式声明
- 研究可以委派，综合理解不能委派
- 验证必须与实现解耦，否则系统会奖励自证正确
- agent 生命周期必须可观测、可中止、可清理
- 真正的并行价值在于职责更清楚

下一章要讨论的，是当这一整套机制落到团队里时，如何从个人技巧变成组织制度。也就是说，`CLAUDE.md`、skills、approval、hook、memory 这些东西，怎样从“某个高手自己会用”变成“一个团队可以稳定复用”的工程实践。

第 8 章团队落地：把一个聪明工具变成可承受的工作流

8.1 个人顺手，不代表团队就能稳定复用

很多 AI coding 工具在个人手里看着很灵。熟练用户知道什么时候该补上下文，什么时候该盯着它别乱动，什么时候一句“不要碰这个目录”就能把风险压住。于是团队很容易产生一个错觉：既然高手已经能把它用顺，那推广不过是多写几篇经验文档。

问题在于，个人技巧之所以有效，恰恰因为它依赖个人持续盯防、背景知识和临场判断。团队一旦接手，问题就变了。你不能再假定每个人都知道哪些命令危险，哪些 memory 已经过期，哪些 skill 会 fork 子代理，哪些步骤必须 ask，哪些步骤只是“看起来没事”。

所以，团队落地真正要解决的，是把原来靠高手脑内维持的秩序，压成多数人都能重复执行的工作流。

Claude Code 的源码之所以值得参考，在于它把很多高手经验显式化了：指令如何分层加载，权限如何决策，子代理如何隔离，生命周期上有哪些可插点。这些实现细节提醒我们，团队采用一个 coding agent，既是在引入一个更聪明的补全工具，也是在重新安排“谁在什么边界内做什么事”。

8.2 团队第一步，是先把最低边界做清楚

这一章最容易被误读的一点，是把团队落地想成“大规模制度化工程”。现实里，多数团队一开始不会先上 hooks、审计链和复杂 skill 目录，他们更常见的起点其实只有四件事：

- 哪些任务允许 agent 直接参与
- 哪些改动必须经过人工 review
- 改完至少要跑什么验证

- 哪些资源一律不能碰

这四件事看起来朴素，却比一堆宏大口号更重要。团队起步阶段更需要的，是先把最低可控边界画清楚。

如果把这个边界画错，后面所有自动化都会失真：

- 允许范围没定义，大家会拿 agent 去做本不该自动化的事
- review 责任没定义，出问题时没人知道最后一层把关是谁
- 验证标准没定义，系统会自动学会迎合最低标准
- 禁区没定义，所谓效率提升最后只是扩大事故半径

因此，更现实的团队落地顺序通常会是：

1. 先把可接受使用范围讲清楚

2. 再把 review 和验证口径讲清楚

3. 然后才考虑如何把高频流程复用起来

很多团队最后失败，往往不是因为 agent 不够强，而是因为起步时跳过了这一步。

8.3 `CLAUDE.md` 的价值，在于稳定、分层、少争议

前面已经讲过 `claudemd.ts` 的分层加载。到了团队采用阶段，这件事仍然重要，但它的意义应该理解得更克制一些。

团队级 `CLAUDE.md` 更适合承载稳定规则，不必把所有流程细节都堆进去。比如：

- 代码库级硬约束，例如禁止写某类目录、禁止危险命令
- 统一验证口径，例如改完至少跑哪些检查
- 协作纪律，例如不要覆盖用户未要求改动，不要在脏工作区擅自 reset
- 输出风格，例如 review 先报 findings，再报总结

不适合堆进去的，通常是这些：

- 会频繁波动的临时流程
- 只有少量任务才用到的操作细节

- 本来更适合沉淀成 command、skill 或脚本的步骤

原因很简单。`CLAUDE.md` 一旦被写成百科全书，它就会迅速失去两个最重要的属性：稳定性和可信度。团队成员不再确定它写的到底是现行规则，还是半年前遗留的讨论；系统也会学会一种很糟糕的行为模式：把过期规范当现行法律。

所以，团队 `CLAUDE.md` 的理想状态不是信息越多越好，而是内容稳定、几乎不需要争论。它更像地基，而不是公告栏。

8.4 复用的重点，先是验证定义，再是 skill 数量

落地 AI coding agent，最常见的失败不在 prompt，不在 model，也不在 skill 数量，而在团队对“完成”根本没有统一定义。

有人觉得能跑就行。有人觉得测试过一半就行。有人觉得模型解释得挺像样也行。这样一来，再聪明的系统也只能学会迎合最低标准。

Claude Code 源码里反复出现的一个倾向，是把 verification 从“顺手看看”提升成独立动作。前面提过 coordinator mode 会把 verification 单独抽出来；`verify` 相关约束也不只是在检查文件存在，更强调要证明改动确实起作用。

这对团队尤其关键，因为 skill 可以复制流程，但只有验证定义才能复制质量。

更现实的团队做法通常是先把下面三件事写清楚：

- 哪些任务必须有独立验证
- 验证至少包含哪些动作，例如测试、运行、日志检查或人工验收
- 验证失败时能不能标记为“已完成但带已知问题”

这三件事如果不清楚，后面任何自动化都只是在加速模糊。相反，只要这三件事先统一了，即使一开始 skill 很少，团队也能先把质量底线稳住。

所以，从落地优先级看，正确顺序通常是：

- 先统一验证定义
- 再把高频流程沉淀成 skill 或命令
- 最后才考虑更复杂的自动化编排

8.5 skill 更适合作为工作流模块来理解

很多团队第一次做 skill，最容易走偏的一点，就是把它当成“长一点的 prompt 模板”，或者反过来，把它神圣化成“组织制度切片”。

这两种理解都不够准确。

从 Claude Code 的实现看，SkillTool 显然不是一个随口参考的提示词仓库。匹配到 skill 时必须实际调用；skill 已经加载过就不应重复加载；必要时它还会在 forked sub-agent context 中执行，有自己的上下文边界和工具集合。

这说明 skill 至少是带执行语义的工作流模块，而不只是文本说明。

但落到团队实践时，更稳妥的理解仍然是：skill 先解决“高频任务如何稳定复用”。至于更重的制度职责，通常没必要一开始就全压在它身上。多数团队真正需要 skill 的场景，通常是这些：

- 某类改动每次都要重复同一组检查
- 某类问题总要查同几份上下文
- 某类任务需要固定输出物
- 某类流程适合分给子代理独立完成

这时 skill 的价值在于把知识、顺序、边界和输出物打包成可重复调用的模块。

所以，团队做 skill 时，更值得先问的是：

- 这个 skill 到底服务哪类任务
- 它默认能动哪些工具
- 它是直接执行，还是应该 fork 给子代理
- 调完以后应该留下什么结果，怎样验证

如果这些问题答不出来，skill 很快就会退化成名字好听、内容冗长、但谁也说不准实际会发生什么的“半自动口号”。

8.6 approval 的重点，是按风险分层

Claude Code 从权限判断到局部 allow rule 注入，始终在强调一件事：会做，不等于应该被允许做。

这一点在个人使用时容易被低估，因为个人往往愿意临场放权。可团队里不一样。一旦代理开始写文件、改 Git 状态、调网络、访问外部系统，它做的每一步都不只是技术动作，也是在移动责任边界。

但多数团队在这里也常犯另一个错误：一上来就想设计非常复杂的审批体系，结果落地成本极高，最后没人执行。

更现实的做法通常是先按风险分层，不必按工具名字一刀切。比如：

- 读文件、列目录、纯分析，通常风险较低
- 改工作区、改配置、执行写操作，风险明显更高
- 推 Git、打外网、访问敏感环境，风险再上一个等级

这种分层方式比“这个工具一律允许、那个工具一律禁止”更接近后果本身。因为团队真正要控制的，是不可逆性和环境敏感度，而不是按钮名称。

所以 approval 在团队落地中的价值，主要是把风险边界说清楚。边界一旦清楚，自动化才不会在错误的地方放大伤害。

8.7 hook 是高级能力，通常不必作为第一步

`hooksConfigManager.ts` 暴露了很多生命周期事件：`SessionStart`、`SessionEnd`、`SubagentStart`、`SubagentStop`、`PreCompact`、`PostCompact`、`FileChanged`、`DirectoryChange` 等。把这些事件放在一起看，会明白 hook 的真正价值：它让制度有机会在正确时机发生。

比如：

- instruction file 加载时补充组织级上下文
- subagent 停止前补一轮验证
- compact 前后记录摘要
- session 结束时做归档或清理

这些都很有用。但更重要的是看清楚一件事：有用，不等于应该优先引入。

对多数普通研发团队来说，更常见的第一步会是：

- 仓库级说明文件
- code review 规则
- CI 和测试要求

- 少量高频命令或 skill

只有当使用规模、风险等级或合规要求继续上升时，hook 才开始真正体现价值。否则它很容易引入新的复杂度：脚本没人维护、触发时机没人说清、调试成本反而比人工操作更高。

因此，更成熟的判断是：hook 属于高级自动化接口，适合挂“时点动作”，也更适合放在基础治理已经稳定之后再引入。

8.8 可复盘轨迹很重要，但要分清基线层和高阶层

这一章最容易被说重的地方，就是“观测与审计”。

事情出错以后，团队当然需要知道为什么发生、从哪一步开始偏离、谁批准了关键动作。这一点没有问题。Claude Code 的实现里，日志、telemetry、task output、transcript path、hook event、agent notification 这些东西拼起来，确实构成了更强的复盘能力。

但现实里必须区分两层：

第一层是多数团队都会有、也已经足够支撑日常复盘的基线轨迹：

- Git diff 和 commit 记录
- PR 评论与 reviewer 意见
- CI 结果与测试日志
- issue、任务单和验收结论

第二层才是更高阶的 agent 过程轨迹：

- transcript path
- tool 调用记录
- hook 事件
- compact 前后摘要
- 子代理 usage 与状态变化

对大多数团队来说，真正的关键通常是先确认第一层没有缺口。因为很多团队连基本的验证和 review 都没统一，过早追求全链路 agent 审计，最后很容易把治理做成高成本摆设。

更合理的说法是：

- 基线层复盘能力，是团队采用 agent 的必需品
- 高阶层审计轨迹，是高风险、高规模、强合规团队的增强项

把这两层分开，才不会把少数平台团队的治理要求误写成所有团队的通用起点。

8.9 从源码里可以提炼出的第八个原则

这一章最后更准确的压缩句，应该是：

> 团队落地的关键，是先把可接受边界、验证标准和高频工作流稳稳固定下来。

Claude Code 的源码给出的启发，真正可迁移的大概是这些：

- 指令要先分层，稳定规则和临时流程不要混在一起
- skill 适合沉淀高频工作流，但前提是适用边界、工具范围和输出物清楚
- approval 应按风险和环境分层，不必按工具名字粗暴切割
- hook 很强，但属于高阶能力，应该在基础治理稳住以后再引入
- 复盘轨迹要分层建设，先保证基线层清楚，再决定是否上高阶审计

如果把它翻成团队可执行原则，大概会更接近下面这几条：

- 先定义可接受使用范围，再谈大规模推广
- 先统一验证定义，再扩 skill 数量
- 先用 review、CI 和最少说明文件把底线稳住，再引入 hooks 和复杂编排
- 任何自动化流程都要能解释它做了什么，但不必一开始就追求全链路重审计
- 团队更值得追求的，不是制度越堆越多，而是边界越清楚、系统越可承受

下一章是全书收束。前面几章一路讲下来，其实是在不断逼近同一个判断：模型是最不稳定的部件，所以真正要设计的，是如何让系统在它不可靠的前提下，仍然输出可承受、可验证、可纠偏的行为。

第 9 章 Harness Engineering 十条原则

写技术书有个坏习惯，讲完一堆细节以后，生怕下判断，仿佛只要把复杂性展示够了，就可以免除结论责任。其实不行。复杂归复杂，判断还是要下。因为团队真正带得走的，往往是若干能反复使用的原则，而不是某个版本的函数名。

前面八章绕来绕去，无非是在逼近这样一个事实：模型不可靠，但系统仍然可以可靠；前提是你别把可靠性寄托在模型身上，而要把它做进 harness。

这一章不再展开细讲，只把前面的论点压成十条原则。它们不是格言，也不是口号，它们来自 Claude Code 源码与工程结构里的工作判断。

9.1 把模型当不稳定部件，不要当同事

同事可以被信任地承担职责，模型不能。模型也许能像同事一样说话，但它不会自动获得同事那种稳定性、责任感和持续判断力。你越早承认这一点，系统就越早开始补上权限、恢复、验证和回滚。

9.2 Prompt 是控制面的一部分

system prompt 用来定义行为协议。它和 runtime、tool schema、memory、hook 一起组成控制平面。把 prompt 当人格设定，最后你会得到一个很会表演、但不受约束的系统。

9.3 Query loop 才是代理系统的心跳

真正的 agent 依赖一段持续的执行循环。输入治理、流式消费、工具调度、恢复分支、停止条件，都是这个心跳的一部分。没有 query loop 的系统，也许能做 demo，但还谈不上运行时。

9.4 工具是受管执行接口

一旦模型开始碰 shell、文件系统、Git 和网络，问题就从“它会不会说”变成“它会不会留下后果”。所以工具必须被调度、被授权、被中断、被补账。越危险的工具，越不能按普通能力对待。

9.5 上下文是工作内存

能塞进上下文，不等于应该塞进去。长期规则、持久记忆、会话连续性和临时对话，应该分层治理。compact 的目标是保住继续工作的语义底座。好的上下文管理，标准不是”够多”，而是”可治理”。

9.6 错误路径就是主路径

prompt too long、max output tokens、中断、hook 回环、compact 自身失败，这些都是长会话代理迟早要面对的日常天气。恢复、熔断、限次、防死循环，必须在设计时就存在，而不是出问题以后再补。

9.7 恢复的目标是继续工作

截断之后最好的动作，通常是续写；压缩失败时最重要的是先让系统恢复呼吸。工程系统真正的礼貌，在于别把用户困在失败态里。

9.8 多代理的意义，是把不确定性分区

多代理会把研究、实现、验证、综合放进不同职责容器里。隔离状态，分离角色，最后由 coordinator 收束理解。并行真正带来的价值，不是更快，而是让职责边界更清楚。

9.9 验证必须独立，不能让系统自己给自己打分

实现者天然倾向于相信自己的改动“差不多行了”。模型更是如此。凡是重要任务，验证都应该成为独立阶段，最好还有独立角色。否则所谓“完成”，很快就会退化成“已经写

完并且我觉得没问题”。

9.10 团队制度比个人技巧重要

一个高手可以靠经验把代理驯服，一个团队不行。团队需要的是：

- 分层 `CLAUDE.md`
- 明确 approval
- 可执行 skill
- 生命周期 hook
- 可追踪 transcript
- 统一验证定义

只有把个人经验制度化，代理系统才可能成为组织能力，而不是个人把戏。

9.11 最后一句话

如果一定要把全书再缩成一句话，那大概就是：

> Harness Engineering 关心的是：在模型并不可靠的前提下，系统仍然能表现出工程系统应有的行为。

Claude Code 的源码真正教人的，是这种克制：它始终把不稳定性当已知前提，再围着这个前提设计 prompt、loop、tools、memory、compact、recovery、verification 和 team workflow。也正因为如此，它值得被当作设计样本。

书写到这里，结论其实已经比过程简单。难的从来不在把原则说出来，而在愿不愿意承认：Harness 比激情重要，制度比聪明重要，验证比自信重要。谁把这三句话听进去了，大概就已经站在 Harness Engineering 的门口了。

附录 A 检查清单：把原则落成能执行的约束

前面几章一直在谈原则。原则如果不能落成检查清单，最后很容易只剩一些听起来都对、却落不了地的判断。附录的任务，就是把那些容易说、难坚持的判断，压成几组可以直接拿去用的清单。

这些清单并不保证系统自动变好，它们只是防止一些最常见、也最无聊的错误反复出现。工程里很多进步，本来就靠少犯重复错误，而不只是靠灵感。

A.1 Agent Runtime 设计清单

一个 AI coding agent 如果要进入真实工程工作流，至少该回答清楚这些问题：

- 是否存在明确的 query loop，而非把每轮调用都当作独立问答
- 是否有跨轮状态对象，明确记录恢复、预算、压缩、hook、turn 计数等信息
- 是否把模型输出当事件流处理，而不只是当最终文案处理
- 是否能在中断时补齐未完成的 tool result，保持执行账本闭环
- 是否区分完成、失败、恢复、继续这些不同终止语义
- 是否为长会话设计了 context budget，而非只在超长时临场补锅

如果这些问题里有两三个答不上来，那么这个系统大概率还停留在“会做 demo”的阶段，离“会跑工程流程”还有距离。

A.2 Prompt 设计清单

system prompt 不该只是长，也该有分层和职责。

检查时至少看这些：

- 是否把身份描述、行为规则、工具约束、输出纪律分开组织
- 是否明确 prompt 的优先级来源，例如默认、项目、自定义、追加、agent 专属 prompt
- 是否把危险动作、越权动作、验证纪律写成明确规则，而非隐约暗示
- 是否避免让 prompt 承担本该由 runtime 处理的职责
- 是否允许团队稳定维护，而非每次修 bug 都往 prompt 里再塞一段话

一个很实用的判断标准是：删掉某段 prompt 以后，系统行为会不会出现结构性变化。如果会，说明它真是控制面；如果不会，可能只是装饰。

A.3 Tool 与 Permission 设计清单

凡是让模型碰世界的系统，都该先问这些：

- 工具调用是否经过统一调度，而非让模型直接裸调
- 并发是否需要显式证明安全，而非默认允许
- 是否存在 `allow / deny / ask` 这样的权限语义分叉
- 高风险工具是否被当成特例治理，而非与普通工具一视同仁
- 是否能对中断、fallback、sibling failure 生成明确收尾语义
- 是否能记录工具执行因果链，避免出现悬空 `tool_use`

如果你的 Bash 和 ReadTool 在治理上几乎一样，那通常说明风险理解得还不够。

A.4 Context 治理清单

任何长会话代理，迟早都会被上下文教育。早点治理，代价比较低。

检查时至少看这些：

- 长期规则、长期记忆、会话连续性、临时对话是否分层
- 是否有明确入口文件和正文文件的区分，避免索引型文件不断膨胀
- 是否对 memory、session memory、skill 附件设 token 预算
- 是否预留 compact 输出空间，而非把窗口吃满再抢救
- compact 之后是否恢复工作语义，例如计划、技能、关键文件、工具状态
- 是否对 compact 自己失败也准备了恢复策略

上下文治理做得好的系统，往往看起来有点吝啬。那种吝啬通常是优点，不是缺点。

A.5 Error Recovery 设计清单

错误恢复最怕两件事：没有设计，和设计成死循环。

至少要检查：

- 可恢复错误是否先进入恢复分支，而非第一时间只展示给用户
- 恢复路径是否分层，先低破坏性，再高破坏性
- 是否有防止 reactive compact、stop hooks、retry 相互咬住的保护
- `max_output_tokens` 后是否优先续写，而非优先 recap
- 自动恢复是否有计数、限次和熔断
- 中断是否也被当作需要语义收尾的失败态

一个不会收手的恢复系统，和一个不会恢复的系统一样危险，只是它危险得更勤奋一点。

A.6 Multi-Agent 设计清单

多代理的重点在于组织不确定性。检查时要看：

- fork 时是否考虑 prompt cache 共享和 cache-safe 参数一致性
- 子代理默认是否隔离 mutable state
- 是否区分 research、implementation、verification、synthesis 角色
- coordinator 是否真正承担综合理解，而非只转发 worker 结果
- verification 是否独立于 implementation
- agent 生命周期是否可观测、可中止、可清理
- 父 abort 是否能传播到子代理，防止孤儿任务残留

如果一个系统号称 multi-agent，但所有 agent 都在做差不多的事，而且没人真正负责 synthesis 和 verification，那它通常只是在把混乱扩成并行。

A.7 Team 落地清单

团队推广时，最容易误判的一点，是把个人熟练度误当成制度成熟度。

落地前最好先核对：

- 是否已有分层 `CLAUDE.md`，且团队知道什么该写进去、什么不该写

- 是否先统一验证定义，再批量造 skill
- 是否按后果和环境敏感度划 approval 边界
- 是否把关键制度挂在合适 hook 时点，而非全塞进静态文档
- 是否保留 transcript、task output、hook event 等复盘证据
- 是否有对 stale memory、过期规则、失效 skill 的维护机制

一个团队真正能承受代理系统，往往是因为普通成员也能在制度内把它用对，而不只是依赖几个高手。

A.8 Review 问题单

如果你要 review 一个 AI coding agent 方案，可以直接追问下面这些问题：

- 哪些行为由 prompt 约束，哪些由 runtime 强制？
- 模型误用工具时，谁来拦？在哪里拦？
- 上下文何时压缩，压缩后如何恢复工作语义？
- prompt too long 和 max output tokens 分别怎么恢复？
- 中断后怎样保证 transcript 和工具结果一致？
- 多代理里谁负责 synthesis，谁负责 verification？
- 失败恢复有没有熔断和防死循环机制？
- 团队如何审计代理做过什么、为什么这么做？

一个方案如果在这些问题上总回答“到时候可以再加”，通常说明它现在还没有真正设计运行时，只是设计了一个乐观场景。

A.9 最后一个清单

如果嫌前面都太长，那至少记住这六条：

- 先设计权限，再设计能力
- 先设计回滚，再设计自治
- 先设计验证，再设计交付
- 先设计上下文预算，再设计长期对话
- 先设计生命周期，再设计多代理
- 先设计制度，再指望团队熟练

做到这六条，系统未必立刻优秀；做不到这六条，系统大概率只是暂时没出事。

附录 B 图示：把运行时骨架画出来

前面的章节一直在用文字解释运行时结构。文字当然能说清楚，但有些东西画成图以后，读者会更快意识到：Claude Code 是一套相当明确的状态机，不只是“一团 prompt 加几个工具”。

B.1 图一：Claude Code 总体控制面

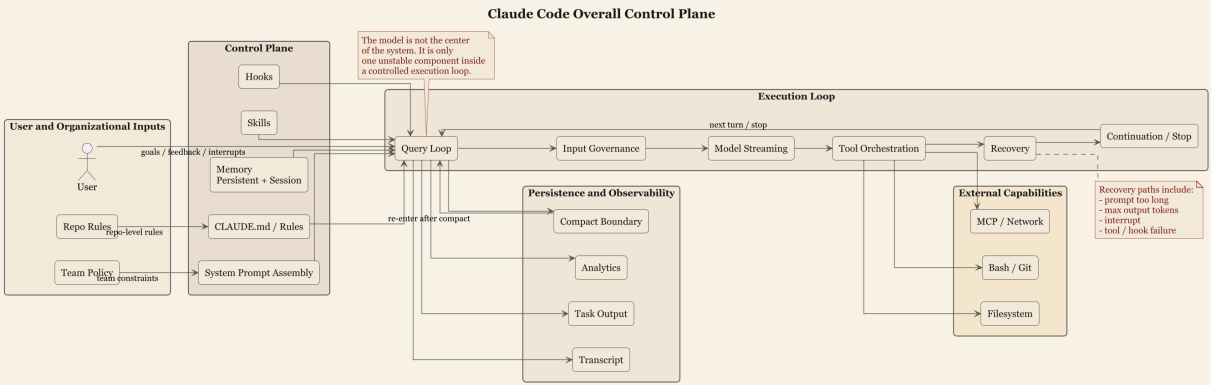

图 9: Claude Code 总体控制面

这张图不该画成“用户 -> 模型 -> 工具 -> 输出”那种幼儿读物，因为那种画法会把真正重要的器官都藏起来。更合理的理解方式，是把 Claude Code 分成五层：

1. 用户交互层
2. 控制面层
3. 执行循环层
4. 外部能力层
5. 持久化与观测层

图的重点是强调以下几点，而不是把模块都列全：

- 模型不在最上层，也不在最底层

- 模型只是 query loop 中的一环
- 真正把系统绑在一起的，是控制面和恢复面

B.2 图二：Query Loop 主循环与恢复分支

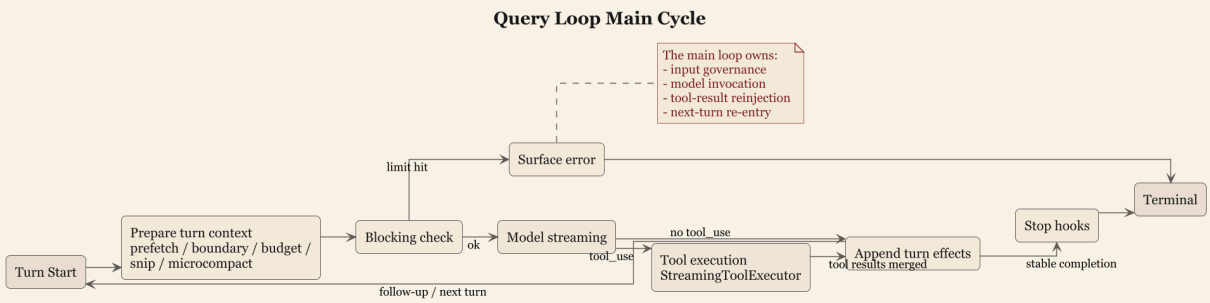

图 10: Query Loop 主循环

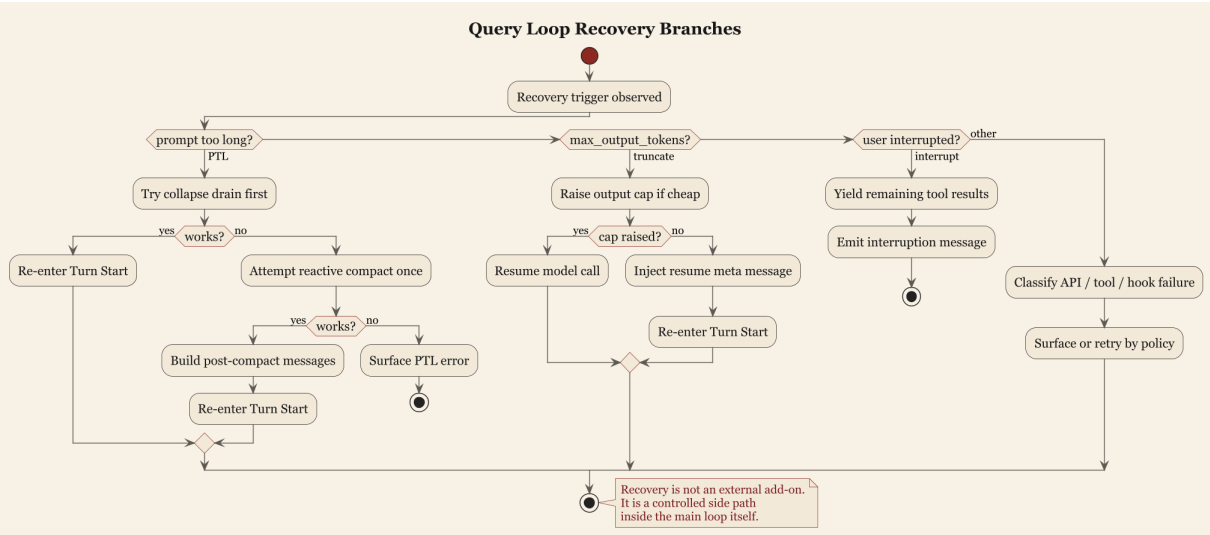

图 11: Query Loop 恢复分支

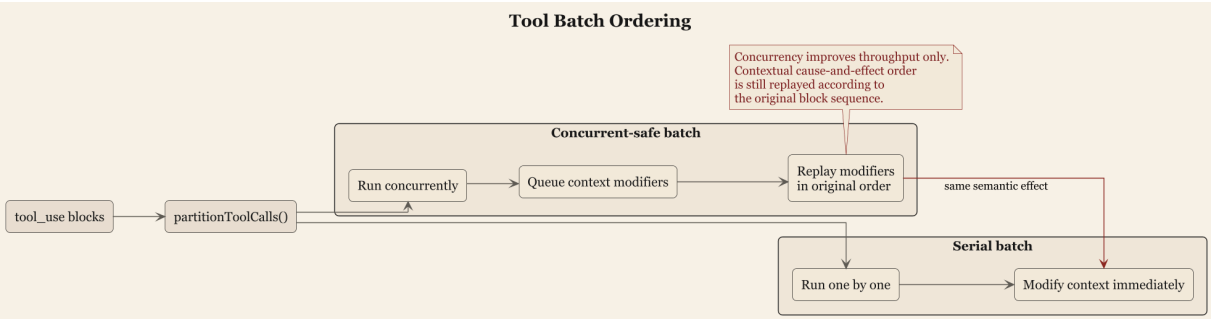

图 12: Tool Batch Ordering

B.3 图三：Tool Batch Ordering 与 StreamingToolExecutor

B.4 图四：Context Sources 与 Compact Rebuild

B.5 图五：Coordinator-Worker Flow 与 Verification Separation

B.6 图六：团队治理图

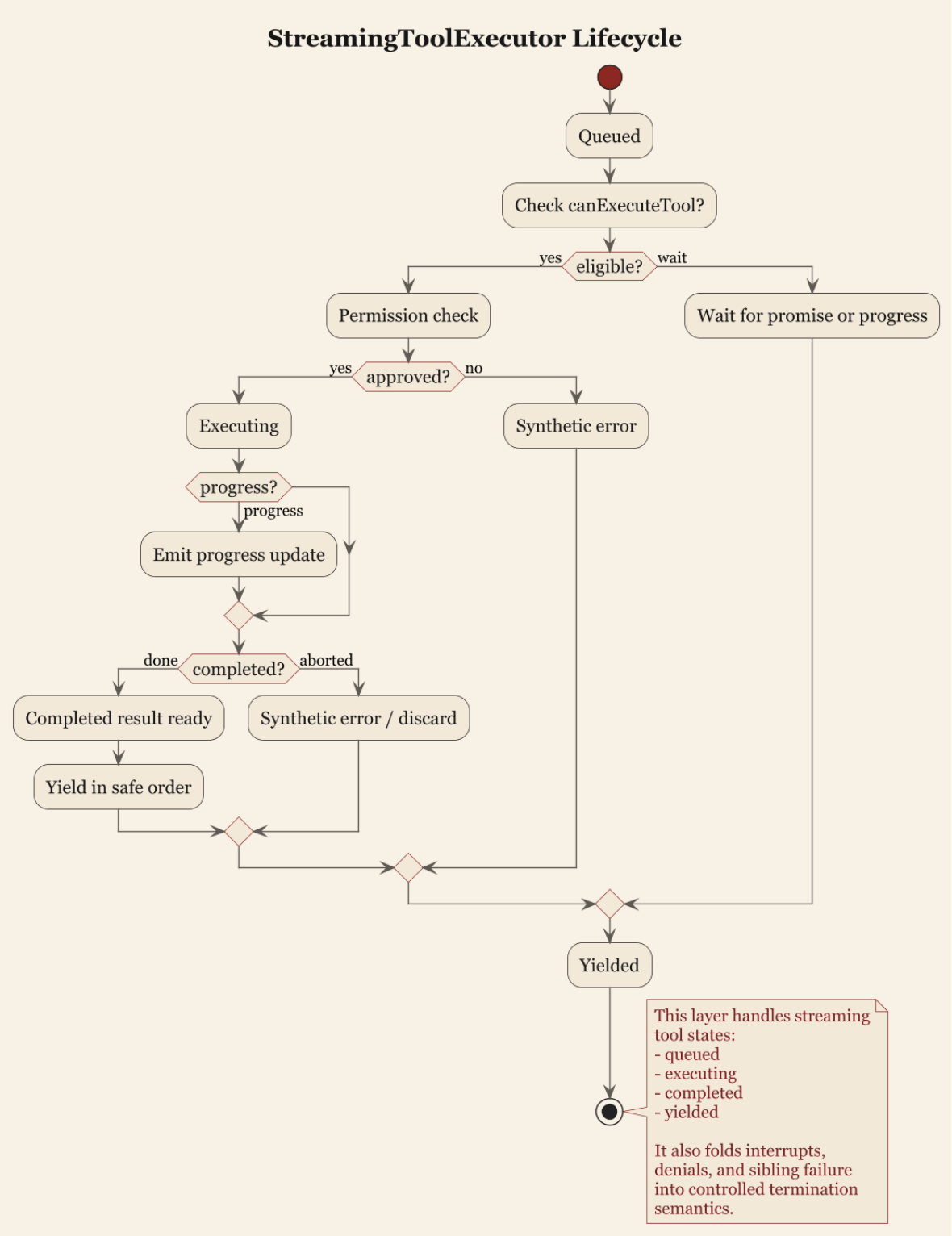

图 13: StreamingToolExecutor Lifecycle

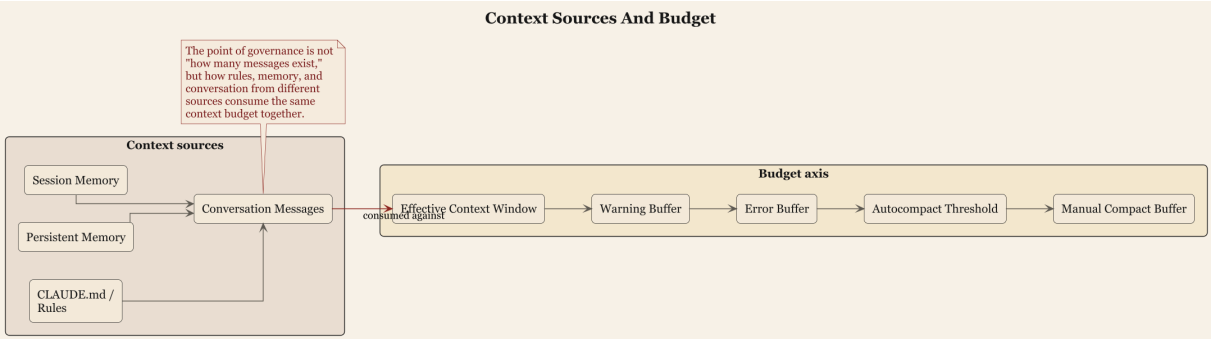

图 14: Context Sources And Budget

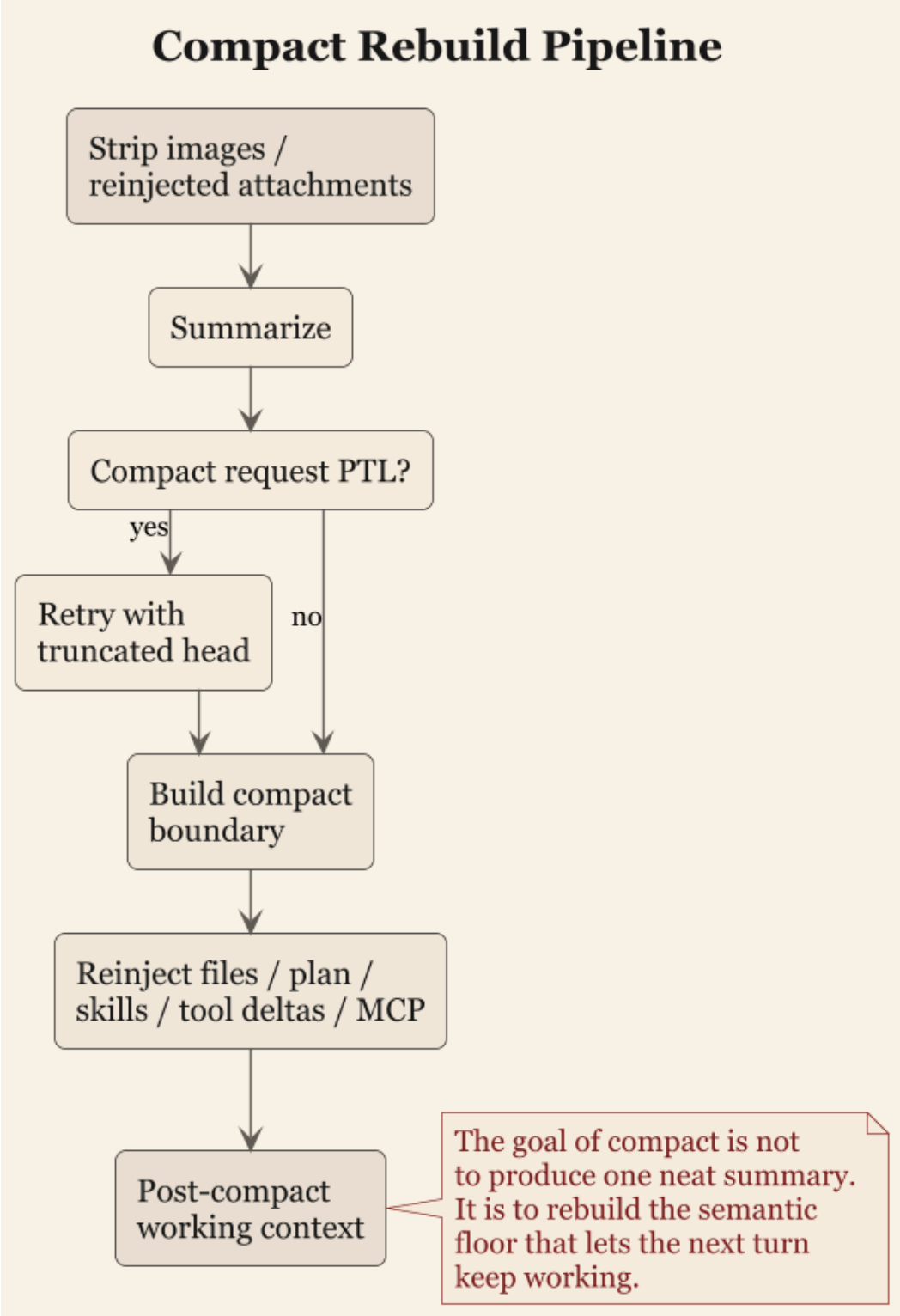

图 15: Compact Rebuild Pipeline

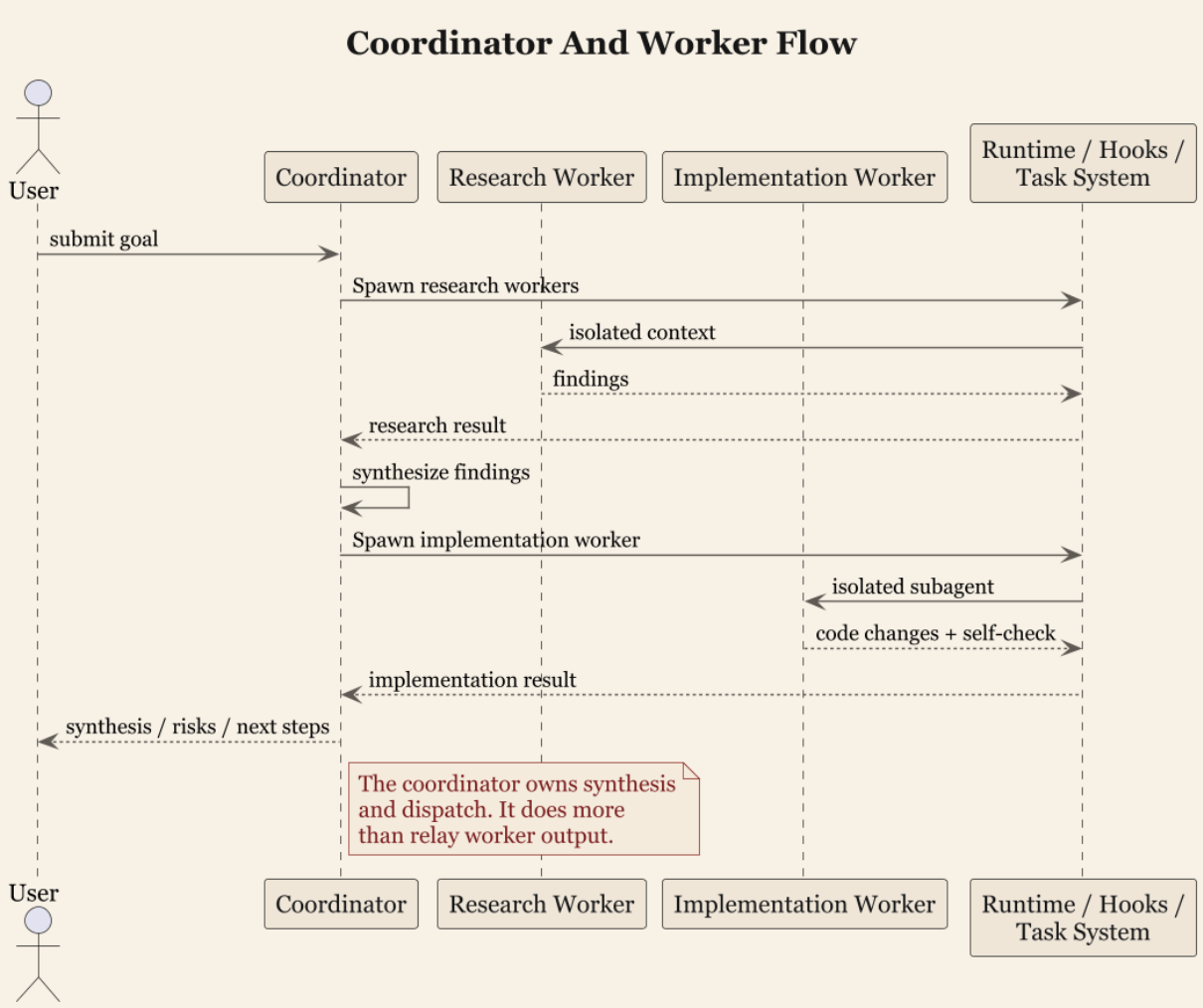

图 16: Coordinator And Worker Flow

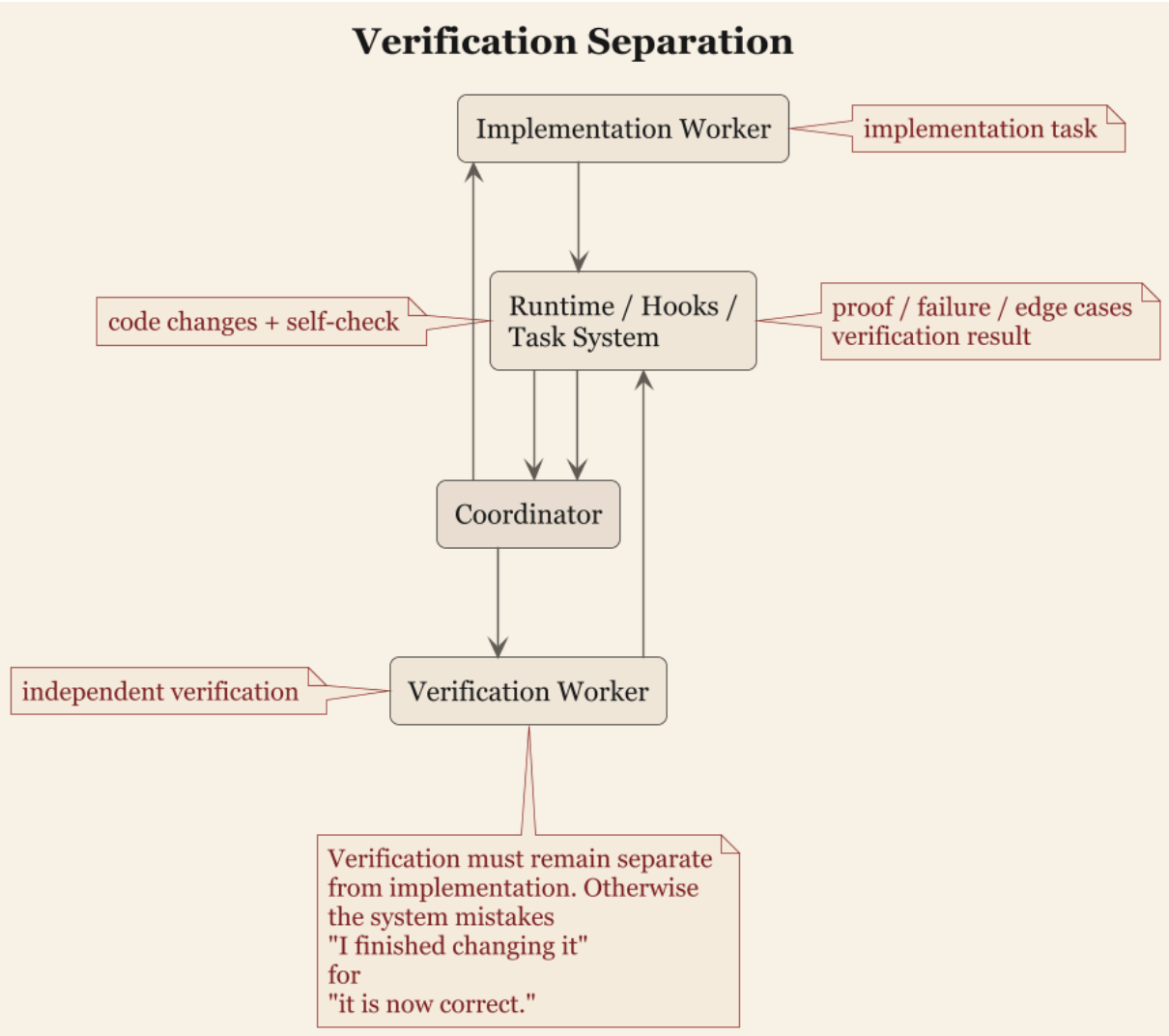

图 17: Verification Separation

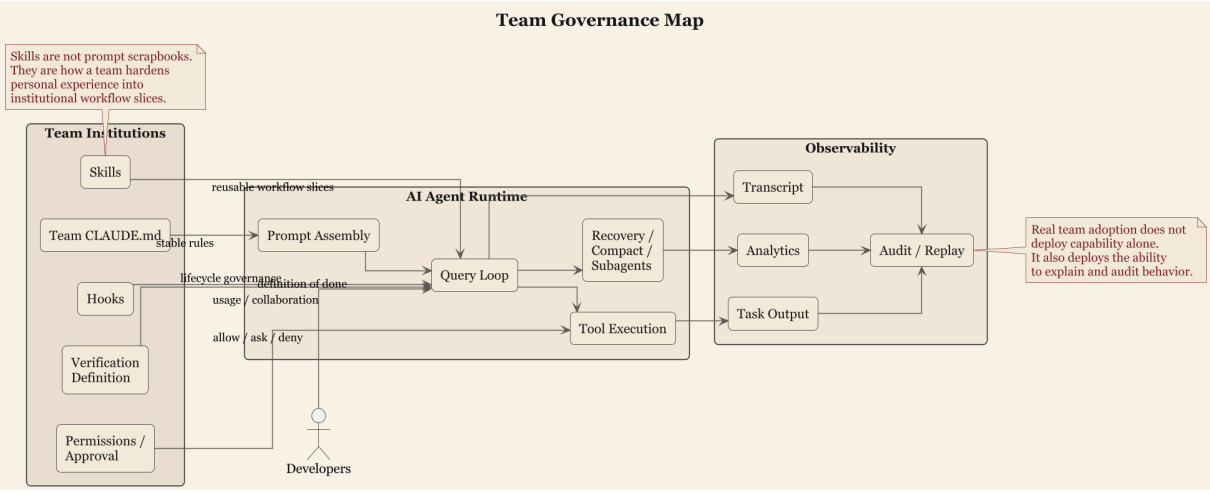

图 18: 团队治理图

附录 C 源码地图：本书各章主要依据哪些文件

这本书虽然不是源码导读，但终究是基于源码写的。这里把每一章最主要的依据文件整理成一份地图。

这份地图不是完整索引，只列与本书论点直接相关的主文件。

这里也补一句版权边界：这份源码地图的作用，是说明分析依据来自哪些文件，而不是承诺随内容提供这些文件的正文内容。这里仅保留必要的工程性引用、模块定位和结构分析，不附源码副本，不做大段实现转载。

C.1 第 1 章为什么需要 Harness Engineering

核心文件：

- `src/constants/prompts.ts`
- `src/utils/systemPrompt.ts`
- `src/query.ts`
- `src/services/tools/toolOrchestration.ts`
- `src/tools/BashTool/prompt.ts`

本章主要论点来源：

- prompt 属于控制面组成部分，而不是人格包装
- query loop 才是代理系统骨架
- 工具和 Bash 风险说明 harness 的必要性

C.2 第 2 章 Prompt 是控制面，不是人格装修

核心文件：

- `src/constants/prompts.ts`
- `src/utils/systemPrompt.ts`
- `src/utils/claudemd.ts`
- `src/memdir/memdir.ts`
- `src/constants/systemPromptSections.ts`
- `src/main.tsx`

本章主要论点来源：

- system prompt 的分层拼装
- `CLAUDE.md` 与 memory 作为控制面输入
- 动态系统提醒与上下文注入

C.3 第 3 章 Query Loop：代理系统的心跳

核心文件：

- `src/query.ts`
- `src/QueryEngine.ts`

本章主要论点来源：

- query loop 的状态机性质
- 输入治理先于模型调用
- 流式事件消费与恢复分支
- QueryEngine 对 conversation lifecycle 的所有权

C.4 第 4 章工具、权限与中断

核心文件：

- `src/services/tools/toolOrchestration.ts`
- `src/services/tools/toolExecution.ts`
- `src/services/tools/StreamingToolExecutor.ts`
- `src/hooks/useCanUseTool.tsx`
- `src/utils/permissions/PermissionResult.ts`
- `src/tools/BashTool/prompt.ts`
- `src/tools/BashTool/bashPermissions.ts`

本章主要论点来源：

- 并发安全与上下文顺序回放
- 工具执行包裹层
- `allow` / `deny` / `ask` 权限语义
- streaming 工具中断与 synthetic result
- Bash 的特殊高压治理

C.5 第 5 章上下文治理：Memory、CLAUDE.md 与 Compact

核心文件：

- `src/utils/claudemd.ts`
- `src/memdir/memdir.ts`
- `src/services/SessionMemory/prompts.ts`
- `src/services/compact/autoCompact.ts`
- `src/services/compact/compact.ts`
- `src/query.ts`

本章主要论点来源：

- `CLAUDE.md` 的分层发现和加载
- `MEMORY.md` 作为入口索引而非正文仓库
- session memory 的结构化连续性
- autocompact 阈值、buffer 与 circuit breaker
- compact 后的工作语义重建

C.6 第 6 章错误与恢复

核心文件:

- `src/query.ts`
- `src/services/compact/autoCompact.ts`
- `src/services/compact/compact.ts`
- `src/services/api/withRetry.ts`

本章主要论点来源:

- withheld recoverable errors
- prompt-too-long 的 collapse drain 与 reactive compact
- `max_output_tokens` 的 escalate 与 resume
- autocompact failure 熔断
- compaction 自身 PTL 重试

C.7 第 7 章多代理与验证

核心文件:

- `src/utils/forkedAgent.ts`
- `src/coordinator/coordinatorMode.ts`
- `src/tasks/LocalAgentTask/LocalAgentTask.tsx`
- `src/utils/hooks/hooksConfigManager.ts`
- `src/skills/bundled/verify.ts`
- `src/memdir/memoryTypes.ts`

本章主要论点来源:

- forked agent 的 cache-safe 参数与状态隔离
- coordinator 的 synthesis 责任
- verification 独立成阶段
- subagent lifecycle hook
- task cleanup 与父子 abort 传播
- 对 stale memory 的 verify discipline

C.8 第 8 章团队落地

核心文件:

- `src/utils/claudemd.ts`
- `src/tools/SkillTool/prompt.ts`
- `src/tools/SkillTool/SkillTool.ts`
- `src/utils/forkedAgent.ts`
- `src/utils/hooks/hooksConfigManager.ts`
- `src/main.tsx`

本章主要论点来源:

- 团队 `CLAUDE.md` 的分层稳定性
- skill 作为制度切片而非 prompt 收藏夹
- approval 边界与 allow rules
- hook 生命周期治理
- session startup 与 instructions / skill loading

C.9 第 9 章十条原则

第 9 章并非直接从单一文件推出，它是对前面所有章节的压缩。它的依据，就是全书已经使用过的这些核心模块共同呈现出的系统结构:

- prompt assembly
- query loop
- tool orchestration
- permission model
- context governance
- recovery system
- multi-agent orchestration
- team governance hooks